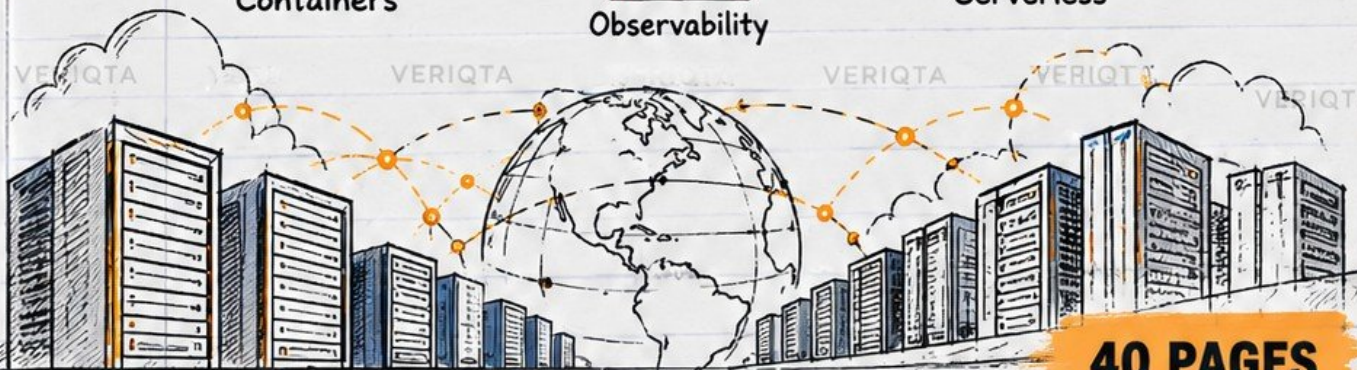


AWS

BLACK BOOK

Advanced AWS Architecture and Operational Knowledge



40 PAGES

AWS Architecture Starts With Failure Domains

Region, Availability Zone, service, instance, dependency, and account boundaries. Designing blast radius before choosing services.

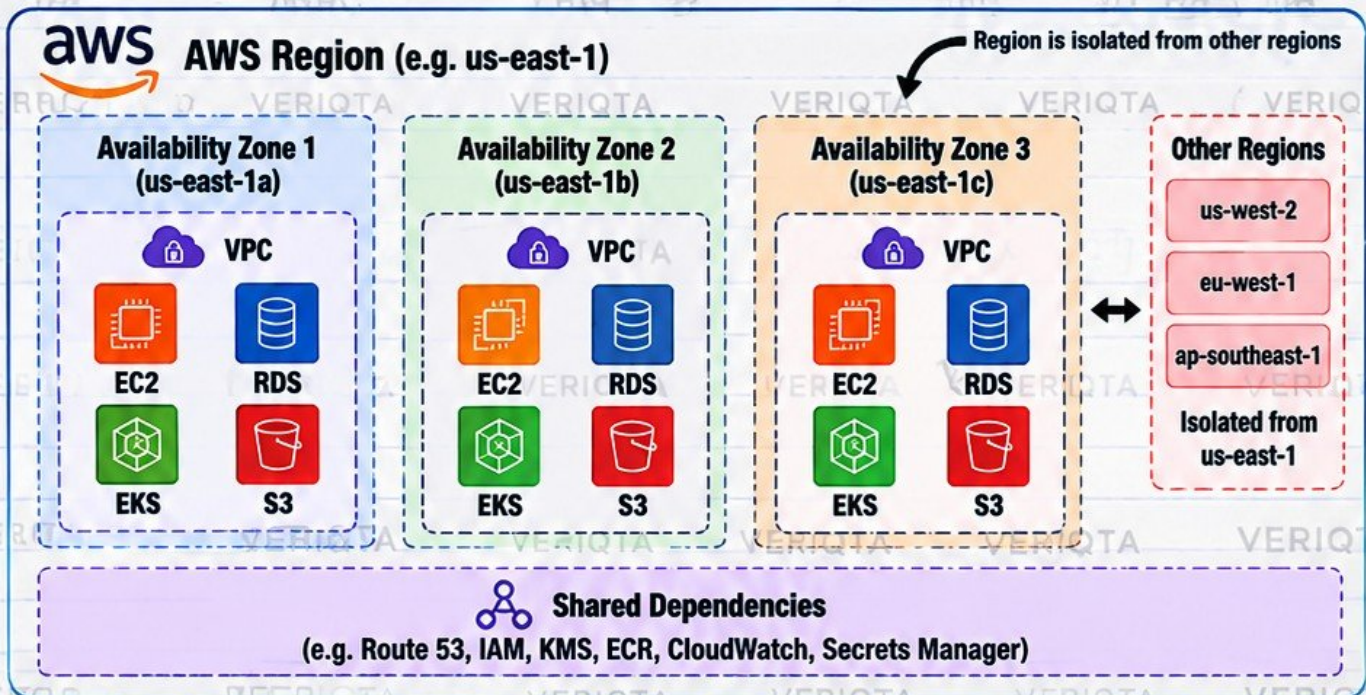
WHY FAILURE DOMAINS MATTER

- Failures are normal in cloud.
- Every AWS service has a failure domain.
- A single failure can impact multiple resources if they share the same domain.
- Understanding failure domains helps you limit blast radius and design for resilience.
- You choose your architecture based on how far a failure should impact your system.

COMMON FAILURE DOMAINS IN AWS

-  **Region** — isolated geographic area
-  **Availability Zone (AZ)** — isolated data center
-  **Service** — managed service control planes
-  **Instance** — individual compute resource
-  **Dependency** — services your workload relies on
-  **Account** — billing, IAM and resource boundary

AWS FAILURE DOMAIN HIERARCHY



DESIGNING BLAST RADIUS

- 1 Identify failure domains for each critical component.
- 2 Decide how far a failure can impact (blast radius).
- 3 Distribute resources across multiple AZs where needed.
- 4 Remove or reduce shared dependencies.
- 5 Use multiple accounts for environment separation.
- 6 Design recovery and fallback for each domain.

KEY TAKEAWAYS

- ✓ A region failure is rare but possible.
- ✓ An AZ can fail at any time.
- ✓ Managed services also have their own failure domains.
- ✓ Your architecture should not place all critical components in one domain.
- ✓ Reducing blast radius improves availability, recovery time and overall resilience.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

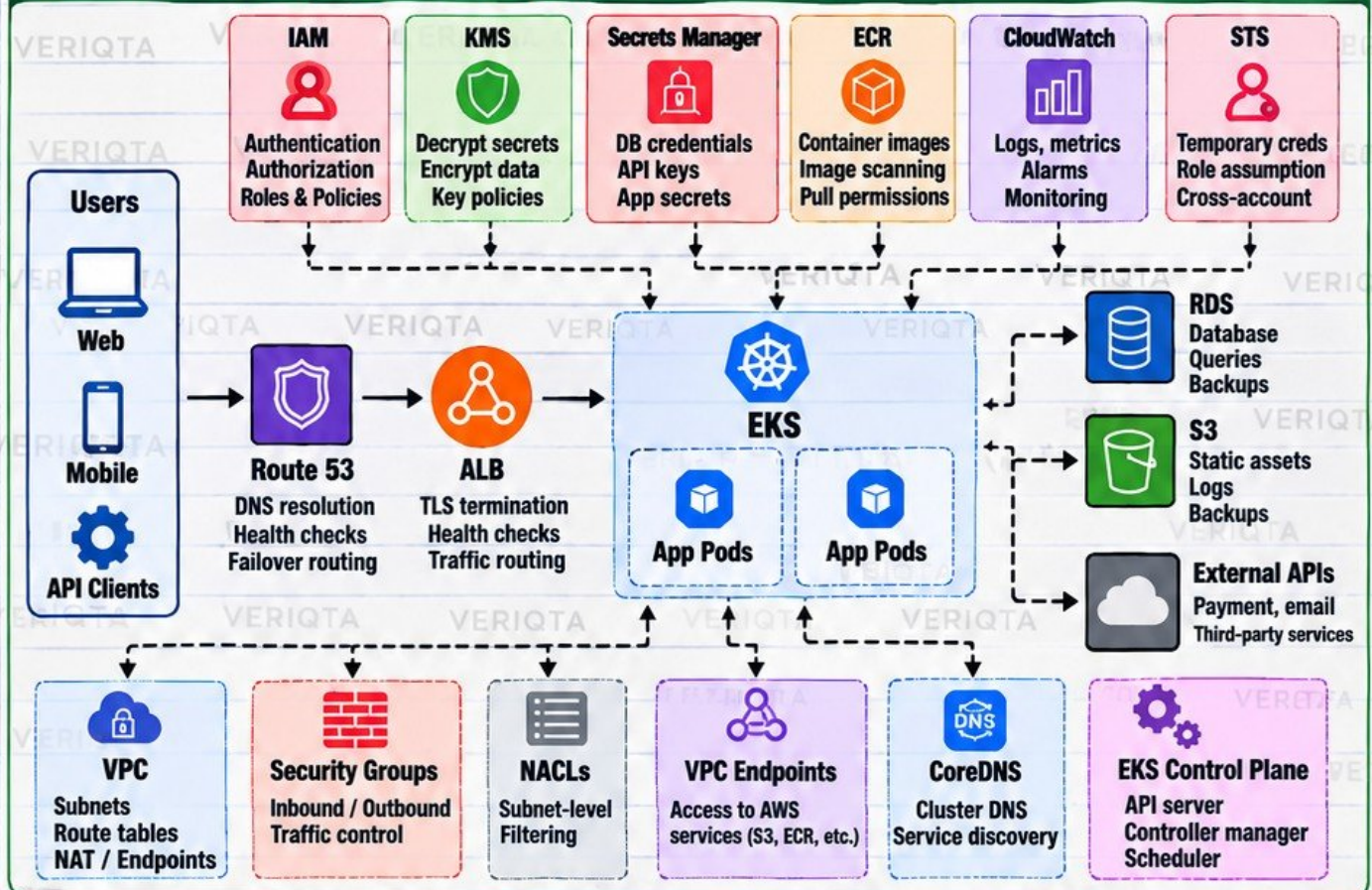


@veriqta

The Hidden Dependency Map

Trace what a production workload actually depends on, including DNS, IAM, KMS, networking, control planes, and downstream services.

EXAMPLE: A WEB APPLICATION ON EKS



KEY DEPENDENCIES TO CHECK

- 1 DNS — Route 53, private hosted zones, VPC DNS
- 2 IAM — roles, policies, service accounts
- 3 KMS — key access and permissions
- 4 Networking — VPC, subnets, security groups, NACLs
- 5 Control Plane — EKS, ECS, or Lambda control services
- 6 Downstream Services — databases, queues, APIs, S3, etc.

WHAT CAN FAIL?

- 1 DNS resolution fails or returns wrong IP.
- 2 IAM permission denied (role or policy issue).
- 3 KMS key access denied or throttling.
- 4 Security group or NACL blocking traffic.
- 5 EKS control plane or regional service issue.
- 6 Downstream service (RDS, S3, external API) unavailable.

TROUBLESHOOTING APPROACH

- 1 Define the blast radius (what is affected?).
- 2 Check recent changes (deployments, config, IAM, DNS).
- 3 Verify each dependency in order (DNS → IAM → KMS → Network → Service).
- 4 Use logs, metrics and events for evidence.
- 5 Isolate the failing dependency.
- 6 Fix, verify, and add prevention (alerts, limits, redundancy).

SENIOR ENGINEER INSIGHTS

- ✓ Map dependencies before production.
- ✓ Reduce unnecessary dependencies.
- ✓ Use VPC endpoints to remove public dependencies.
- ✓ Apply least privilege for IAM and KMS.
- ✓ Design for graceful degradation when downstream services fail.
- ✓ Continuously validate critical dependencies with monitoring and synthetic checks.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

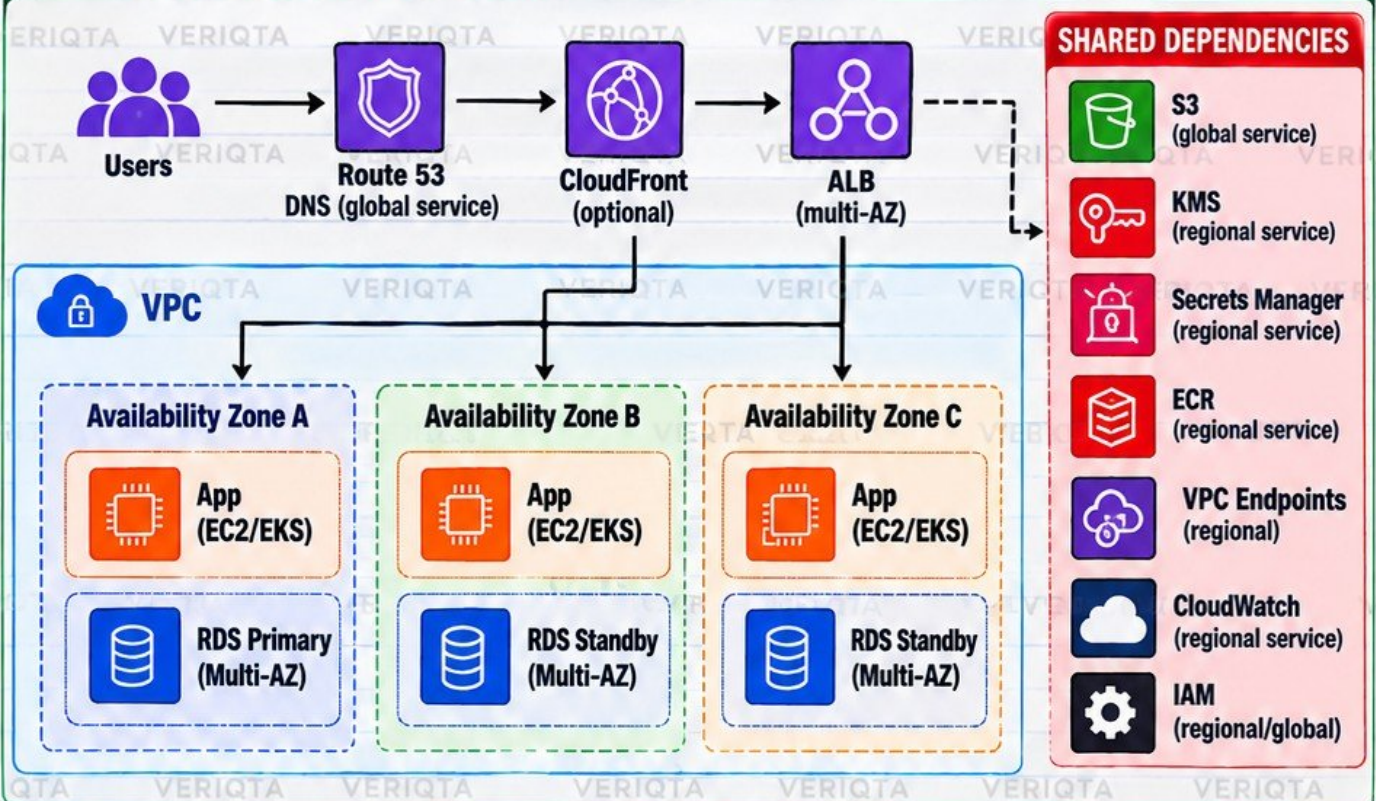


@veriqta

Multi-AZ Is Not Automatically Highly Available

Why distributing resources across AZs can still leave shared dependencies and correlated failure paths.

COMMON MULTI-AZ ARCHITECTURE



WHY THIS IS NOT AUTOMATICALLY HIGHLY AVAILABLE

- 1 Many AWS services are regional, not zonal (e.g. S3, KMS, IAM).
- 2 A regional service outage can impact resources in all AZs.
- 3 Shared dependencies create a single failure domain.
- 4 Network misconfiguration can affect multiple AZs.
- 5 Application design, not just AZ distribution, determines real availability.

EXAMPLES OF CORRELATED FAILURE PATHS

- 1 KMS issue → app in all AZs cannot decrypt data.
- 2 IAM issue → all instances lose permissions.
- 3 S3 or ECR outage → deployments fail across all AZs.
- 4 Route 53 or CloudFront issue → traffic cannot reach any AZ.
- 5 VPC misconfiguration → no connectivity across AZs.
- 6 RDS control plane issue → failover or recovery affected.

REAL WORLD EXAMPLE

An e-commerce platform runs across 3 AZs. All app instances are healthy, but the application cannot read from S3 because of a regional KMS issue.

Result: the entire platform is unavailable across all AZs even though compute is distributed.

SENIOR ENGINEER TAKEAWAYS

- ✓ Identify regional dependencies before production.
- ✓ Map the full dependency chain, not just compute.
- ✓ Design for regional service failures.
- ✓ Use fallbacks and graceful degradation.
- ✓ Test recovery and failover for shared services.
- ✓ Validate IAM, KMS, DNS, and networking as part of your resilience plan.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

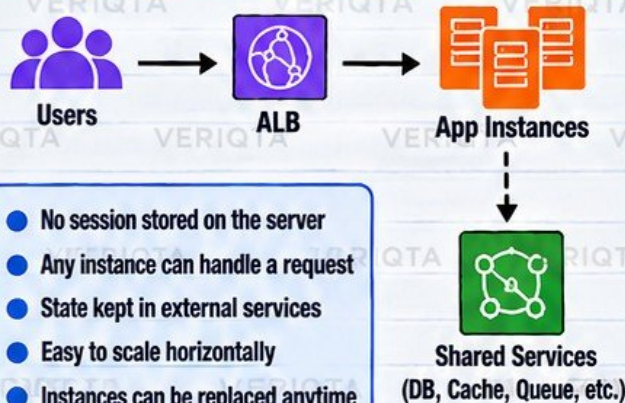


@veriqta

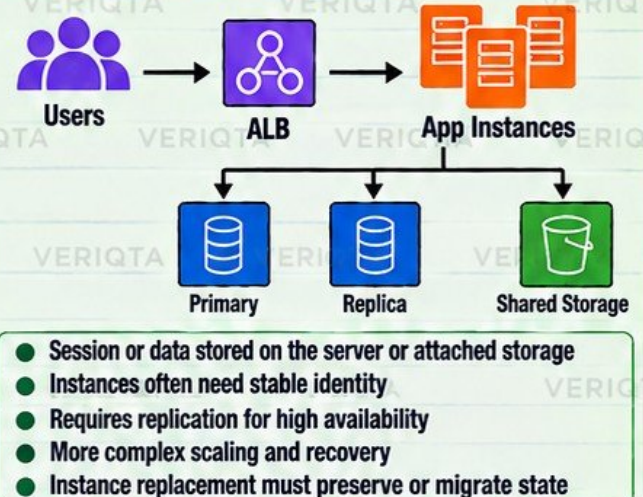
Stateful vs Stateless Architecture at Scale

Placement, replication, session state, recovery, horizontal scaling, and the operational cost of state.

STATELESS ARCHITECTURE



STATEFUL ARCHITECTURE



KEY DIFFERENCES

Aspect	Stateless	Stateful
Session State	Not stored on server (use external store)	Stored on server or attached storage
Scaling	Easy horizontal scaling	More complex, may require data sync
Instance Replacement	Can be replaced anytime	Must preserve or migrate state
Data Persistence	External (RDS, DynamoDB, ElastiCache, etc.)	Local storage or replicated storage (EBS, EFS, etc.)
Placement	Any AZ or Region	Often AZ-aware or needs specific placement
Failure Impact	Lower blast radius	Data loss or service disruption if replication fails
Operational Cost	Lower	Higher (replication, backups, data management)
Examples	Web apps, APIs, microservices, Lambda	Databases, message brokers, caches, StatefulSets

WHERE TO USE STATELESS

- ✓ Web applications
- ✓ APIs and microservices
- ✓ Serverless workloads (Lambda)
- ✓ Containerized workloads (ECS/EKS)
- ✓ Workloads with external data stores
- ✓ When you need fast horizontal scaling
- ✓ When instance identity does not matter

WHERE TO USE STATEFUL

- ✓ Relational databases (RDS, Aurora)
- ✓ NoSQL databases (DynamoDB with state)
- ✓ Distributed caches (ElastiCache, Redis)
- ✓ Message brokers (Kafka, RabbitMQ)
- ✓ File storage (EFS, FSx)
- ✓ Stateful applications (e.g. Elasticsearch)
- ✓ When data consistency and durability are critical

SENIOR ENGINEER INSIGHTS

- ✓ Separate compute from state where possible.
- ✓ Use managed services for stateful components.
- ✓ Design for AZ failures (replication + backups).
- ✓ Understand the recovery time and recovery process for stateful systems.
- ✓ Consider latency, consistency, and cost trade-offs.
- ✓ Use immutable infrastructure for stateless workloads.
- ✓ Test failover and data recovery regularly.

REAL WORLD EXAMPLE

A web application (stateless) runs on EKS across multiple AZs, while the user data is stored in Aurora (stateful) with Multi-AZ replication. If an app instance fails, traffic is routed to healthy instances. If the primary database fails, Aurora promotes a standby and the application reconnects automatically.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Synchronous vs Asynchronous AWS Systems

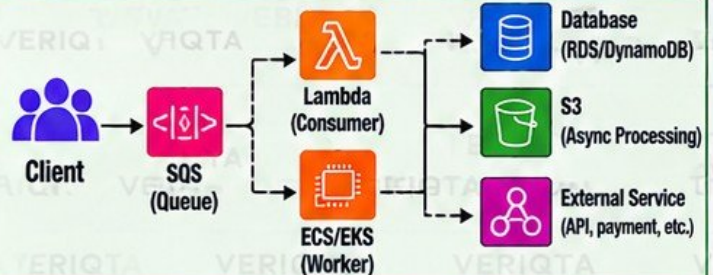
Latency, queues, backpressure, retries, ordering, failure isolation, and when asynchronous architecture becomes necessary.

SYNCHRONOUS ARCHITECTURE



- Client waits for a response.
- Request flows directly to the service.
- Low latency when everything is healthy.
- Failures in downstream services immediately impact the client.
- Suitable for real-time and user-facing requests.

ASYNCHRONOUS ARCHITECTURE



- Client sends a request and does not wait for processing.
- Work is handled by consumers in the background.
- Improves resilience and absorbs traffic spikes.
- Failures are isolated and can be retried.
- Suitable for non-real-time workloads and long-running tasks.

KEY DIFFERENCES

Aspect	Synchronous	Asynchronous
Client Experience	Waits for a response	Immediate acknowledgement
Latency	Low (if healthy)	Higher (eventual processing)
Coupling	Tightly coupled	Loosely coupled
Failure Impact	Downstream failure affects client	Failures isolated from client
Scalability	Limited by downstream capacity	Handles spikes with queues and workers
Use Cases	Real-time, user-facing requests	Background jobs, event processing, integration
Ordering	Natural request order	Must design for ordering if required
Retries	Client or service retries (can amplify load)	Built-in retries with DLQ (SQS) or event replay
Backpressure	Harder to handle, can cause timeouts	Queues absorb pressure and smooth traffic
Operational Complexity	Lower	Higher (monitoring, retries, idempotency)

WHEN TO USE SYNCHRONOUS

- ✓ User-facing APIs and web requests.
- ✓ Low latency requirements.
- ✓ Simple workflows.
- ✓ When downstream services are highly available and reliable.
- ✓ When you need an immediate result.

WHEN TO USE ASYNCHRONOUS

- ✓ Long-running tasks (e.g. video processing, report generation).
- ✓ Non-critical background work.
- ✓ Integrations with unreliable services.
- ✓ High traffic or unpredictable load.
- ✓ When you need failure isolation and better resilience.

SENIOR ENGINEER INSIGHTS

- Retries without backoff can overload downstream systems.
- Design for idempotency to handle retries safely.
- Use dead letter queues (DLQ) for failed messages.
- Monitor queue depth and message age.
- Consider ordering requirements and use FIFO queues when needed (SQS FIFO).

REAL WORLD EXAMPLE

An e-commerce platform uses synchronous calls for product search and checkout, but uses asynchronous processing for order fulfillment, email notifications, and inventory updates. If the payment service is slow, the request is queued and retried instead of blocking the user.



@veriqta



@veriqta



@veriqta



@veriqta



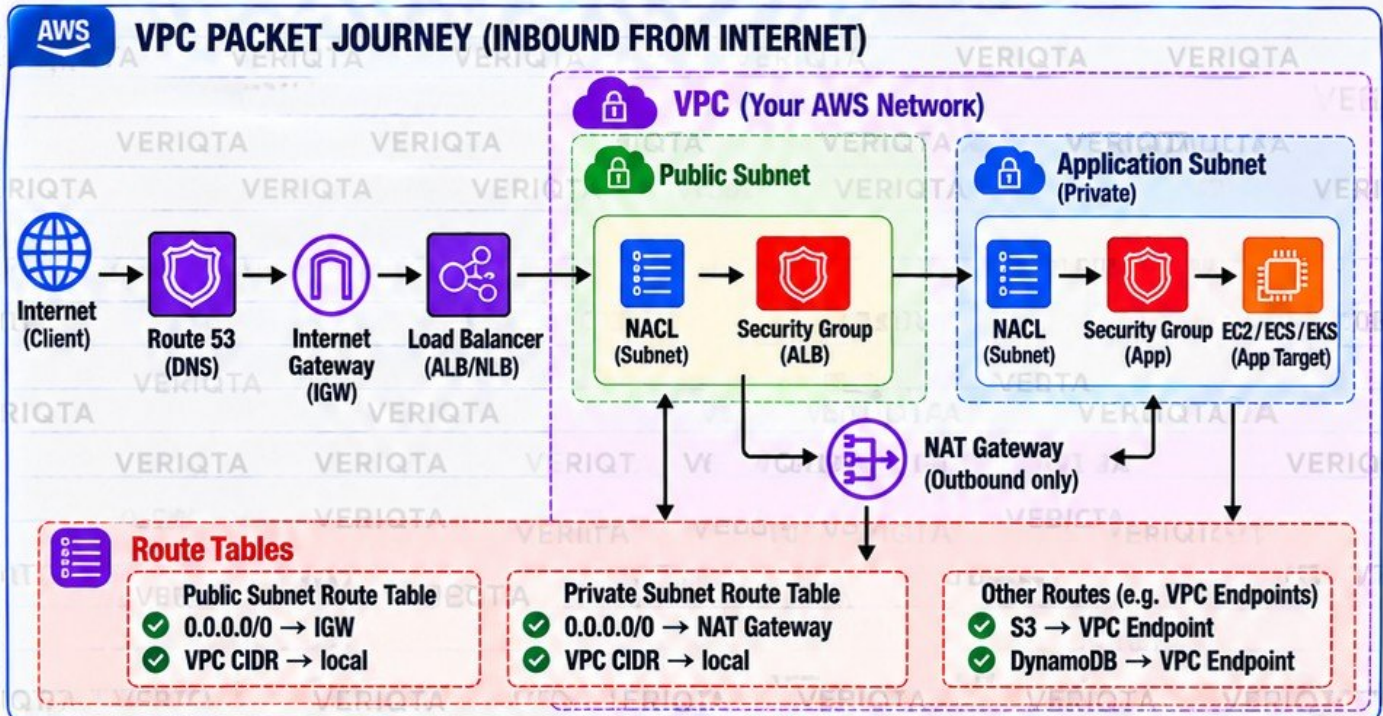
@veriqta



@veriqta

VPC Packet Journey

Follow a request through ENI, route tables, security groups, NACLs, NAT, load balancers, and application targets.



PACKET JOURNEY STEPS (INBOUND)

- 1 Client resolves DNS in Route 53 to the ALB/NLB.
- 2 Request enters AWS through the Internet Gateway (IGW).
- 3 Route table in the public subnet sends traffic to the ALB.
- 4 NACL (public subnet) allows the traffic (stateless).
- 5 Security group (ALB) allows the traffic (stateful).
- 6 ALB forwards the request to targets in private subnets.
- 7 NACL (private subnet) allows the traffic (stateless).
- 8 Security group (instance/service) allows the traffic.
- 9 Request reaches the application (EC2, ECS, or EKS).
- 10 Response follows the same path back to the client.

KEY COMPONENTS AND ROLE

	Route 53	Resolves domain name to ALB/NLB IP.
	Internet Gateway	Entry and exit point for internet traffic.
	Route Table	Decides where traffic should go.
	NACL	Stateless subnet-level filtering (allow/deny).
	Security Group	Stateful instance/ENI-level filtering.
	NAT Gateway	Allows private subnet to reach the internet.
	Load Balancer	Distributes traffic to multiple targets.
	ENI	Network interface for instances and services.
	Application	EC2, ECS, or EKS that handles the request.

OUTBOUND TRAFFIC (FROM PRIVATE SUBNET)

- 1 Instance sends request (e.g. to download updates).
- 2 Route table sends traffic to NAT Gateway.
- 3 NACL (private subnet) allows outbound traffic.
- 4 Security group allows outbound traffic.
- 5 NAT Gateway translates the private IP to a public IP.
- 6 Traffic goes to the internet and response returns via NAT.

COMMON ISSUES AND TROUBLESHOOTING

- 1 No response → check route tables and IGW/NAT.
- 2 Connection timeout → check NACLs and security groups.
- 3 ALB unhealthy → check target health and SG rules.
- 4 Instance cannot reach internet → check NAT and routes.
- 5 DNS not resolving → check Route 53 and VPC DNS settings.
- 6 Intermittent failures → check NACLs (stateless) and return traffic.

SENIOR ENGINEER TAKEAWAYS

- ✓ Understand the full packet path for both inbound and outbound traffic.
- ✓ Always check route tables, NACLs, and security groups in order.
- ✓ Use VPC flow logs to verify traffic and find where it is blocked.
- ✓ Design for least privilege and proper segmentation.
- ✓ Use VPC endpoints to reduce NAT usage and improve security.
- ✓ Document the expected flow for every critical application.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Private Subnets Are Not a Security Strategy

What private addressing actually provides, and where IAM, egress control, endpoints, segmentation, and inspection matter.

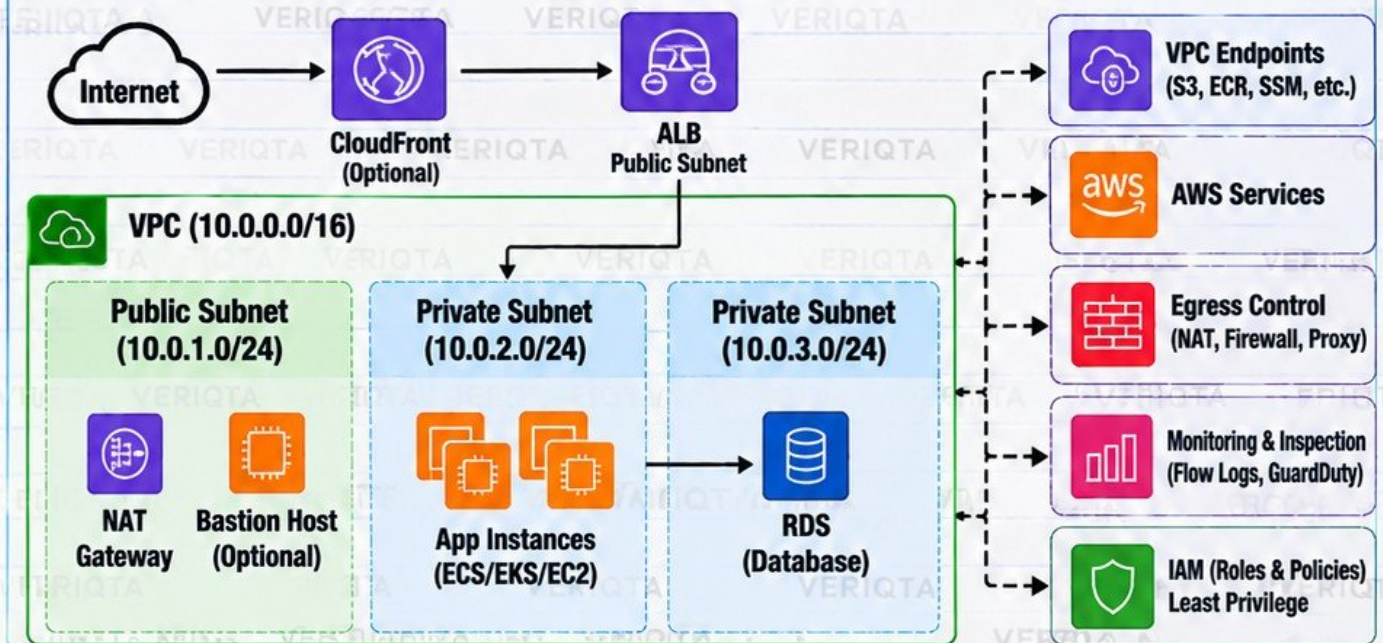
WHAT PRIVATE SUBNETS PROVIDE

- ✓ Use private IP addresses (not publicly routed).
- ✓ Not directly reachable from the internet.
- ✓ Require internal routing (VPC).
- ✓ Commonly used for application and database tiers.
- ✓ Reduces accidental exposure.

WHAT PRIVATE SUBNETS DO NOT PROVIDE

- ✗ They do not block outbound internet access.
- ✗ They do not enforce IAM permissions.
- ✗ They do not inspect or filter traffic.
- ✗ They do not stop lateral movement within the VPC.
- ✗ They do not protect against compromised instances.
- ✗ They do not provide data encryption.
- ✗ They do not replace network segmentation, firewalls, or monitoring.

EXAMPLE: VPC ARCHITECTURE WITH PROPER SECURITY CONTROLS



WHERE REAL SECURITY COMES FROM

- 1 IAM – control what workloads can do.
- 2 Network segmentation – separate tiers and workloads.
- 3 Egress control – NAT, proxy, or firewall.
- 4 VPC endpoints – access AWS services privately.
- 5 Inspection – flow logs, GuardDuty, IDS/IPS, firewall.
- 6 Encryption – in transit and at rest (KMS).
- 7 Monitoring – logs, metrics, alerts.

KEY TAKEAWAYS

- ✓ Private subnets provide private addressing, not security.
- ✓ Use IAM, network controls, and inspection to secure workloads.
- ✓ Limit and monitor outbound internet access.
- ✓ Use VPC endpoints to keep traffic private.
- ✓ Segment workloads to reduce blast radius.
- ✓ Assume a compromised instance can move within the VPC.
- ✓ Layer multiple controls for real security.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

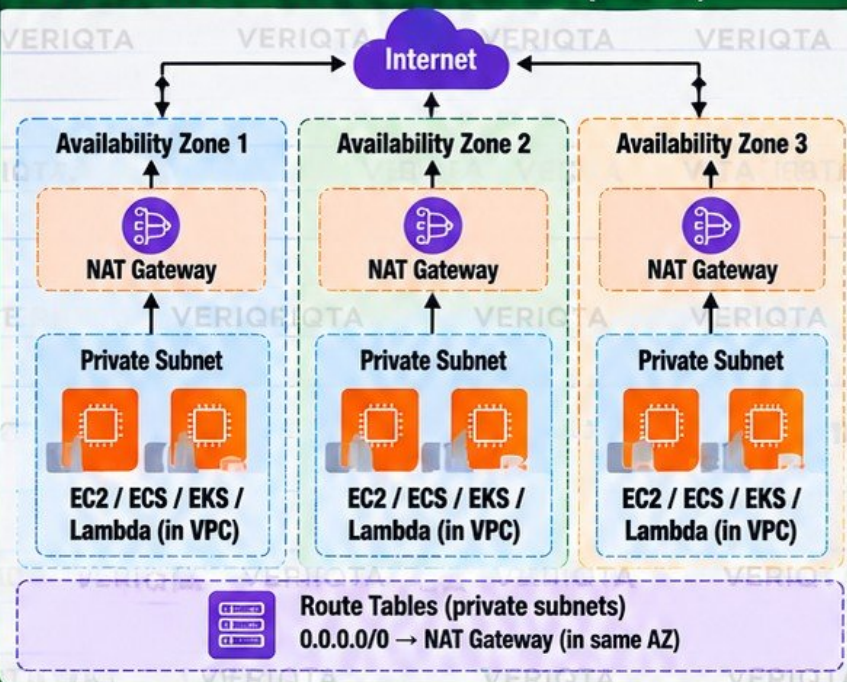


@veriqta

NAT Gateway Architecture at Scale

Availability, AZ affinity, routing, cost, port exhaustion, centralized egress, and failure consequences.

TYPICAL NAT GATEWAY ARCHITECTURE (PER AZ)



KEY POINTS

- 1 NAT Gateway is AZ-specific.
- 2 Private subnets should route to a NAT in the same AZ.
- 3 Used for outbound internet access only.
- 4 Managed by AWS and scales automatically.
- 5 Each NAT Gateway has a public IP (Elastic IP).

USE CASES

- ✓ Outbound internet access for instances in private subnets.
- ✓ Pull container images (ECR).
- ✓ Install OS and application updates.
- ✓ Access external APIs and SaaS tools.
- ✓ Send logs and metrics to third-party services.

ROUTING CONSIDERATIONS

- 1 Each private subnet routes 0.0.0.0/0 to a NAT Gateway in the same AZ.
- 2 If a subnet uses a NAT in another AZ, traffic incurs cross-AZ charges and becomes a single point of failure.
- 3 Return traffic is handled automatically by NAT (stateful).
- 4 Use route tables and subnet design to control egress paths.

COST CONSIDERATIONS

- \$ You pay per hour for each NAT Gateway.
- \$ You pay per GB of data processed.
- \$ Cross-AZ traffic to a NAT Gateway increases cost.
- \$ Centralized egress through a single NAT can become expensive at scale.
- \$ Consider alternatives for high-volume or internal traffic (e.g. VPC endpoints, PrivateLink).

PORT EXHAUSTION (SNAT LIMITS)

- ! Each NAT Gateway has a limited number of ephemeral ports per destination (≈ 55,000 per unique destination IP:port).
- ! High outbound connections to the same destination can exhaust ports.
- ! Symptoms: connection timeouts, intermittent failures, new connections blocked.
- ! Mitigation: distribute traffic, use multiple NAT Gateways, use VPC endpoints, or optimize connection reuse.

FAILURE CONSEQUENCES

- ✗ If a NAT Gateway fails, instances in that AZ lose outbound internet access.
- ✗ If all NAT Gateways are in one AZ, a single-AZ failure affects the entire workload.
- ✗ Health of NAT Gateway is managed by AWS, but route table misconfiguration can still cause outages.
- ✗ Dependency on NAT affects updates, image pulls, external APIs, and monitoring.

BEST PRACTICES

- ✓ Deploy a NAT Gateway in each AZ that has private subnets.
- ✓ Keep subnet routing to the NAT in the same AZ.
- ✓ Use VPC endpoints for AWS services (S3, ECR, CloudWatch, etc.).
- ✓ Use egress control (NAT + firewall/proxy) when required.
- ✓ Monitor connections, errors, and data processed.

ALTERNATIVES AND COMPLEMENTS

- ⚙ VPC Endpoints — private access to AWS services.
- ⚙ NAT Instance — lower cost but you manage scaling and HA.
- ⚙ AWS Network Firewall / third-party firewall — inspect and control egress.
- ⚙ Proxy — for fine-grained egress control.
- ⚙ PrivateLink — access third-party services privately.

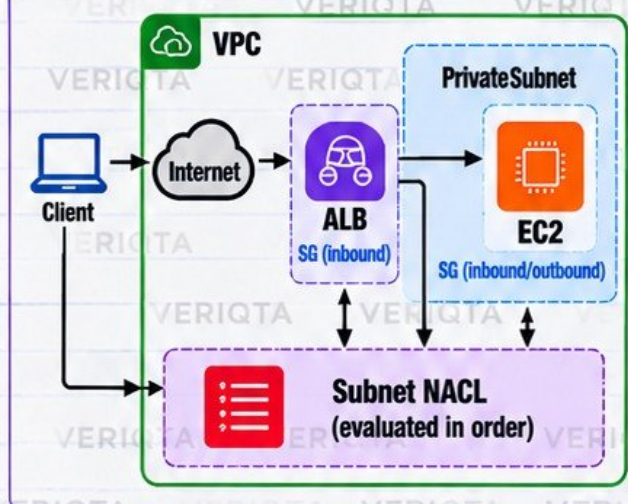
Security Groups vs NACLs Under Failure

Stateful versus stateless filtering, return traffic, rule evaluation, and systematic connectivity debugging.

KEY DIFFERENCES

Security Groups (SG)	Network ACLs (NACL)
Stateful (return traffic allowed)	Stateless (must allow both directions)
Applied at instance/ENI level	Applied at subnet level
Allow rules only (no explicit deny)	Allow and Deny rules
Evaluated as a set (no order)	Evaluated in order (lowest number first)
Commonly used for instance access	Used for subnet-level filtering
Recommended for workload control	Used for additional layer and compliance

HOW TRAFFIC FLOWS



RETURN TRAFFIC BEHAVIOR

Security Group (Stateful)	Network ACL (Stateless)
- You only need to allow inbound traffic on the port (e.g. 80/443). - Return traffic is automatically allowed. - No rules required for the response.	- You must allow inbound AND outbound traffic. - If the response rule is missing, return traffic will be blocked. - Rules are evaluated in order (lowest number first).

RULE EVALUATION EXAMPLE (NACL)

Rule #	Type	Protocol	Port Range	Action
100	All	All	All	Deny
110	All	TCP	443	Allow
120	All	TCP	1024-65535	Allow
130	All	All	All	Deny

- Rules are evaluated in order (lowest number first).
- Add explicit allow rules for return traffic (ephemeral ports).
- End with a deny rule.

COMMON FAILURE SCENARIOS

- Instance cannot be reached: SG inbound rule missing.
- Connection times out: NACL blocking return traffic.
- Intermittent failures: NACL rule order or ephemeral port range.
- Works in one subnet but not another: different NACL rules.
- ALB health checks fail: SG or NACL blocking traffic.
- Asymmetric routing: NACL blocking traffic in one direction.

SYSTEMATIC DEBUGGING STEPS

- Identify the source, destination, port and protocol.
- Check Security Group rules (inbound and outbound).
- Check NACL rules on the subnet (both directions and rule order).
- Verify route tables and network path (VPC, TGW, peering, etc.).
- Test with tools (curl, telnet, nc) from the correct source.
- Review VPC Flow Logs for REJECT entries.
- Fix the rule and retest from the same source.

PRACTICAL EXAMPLE: ALLOW HTTPS TO EC2 IN PRIVATE SUBNET

Security Group (on EC2)	NACL (on subnet)
Inbound: TCP 443 from ALB SG Outbound: All (default)	Inbound: TCP 443 from ALB subnet Outbound: TCP 1024-65535 to ALB subnet Other traffic: Deny

KEY TAKEAWAYS

- Security Groups are stateful, NACLs are stateless.
- NACLs must allow both request and response traffic.
- Use SGs for workload-level control and NACLs for subnet-level and compliance requirements.
- Always verify rule order, ephemeral ports and VPC Flow Logs when debugging connectivity issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

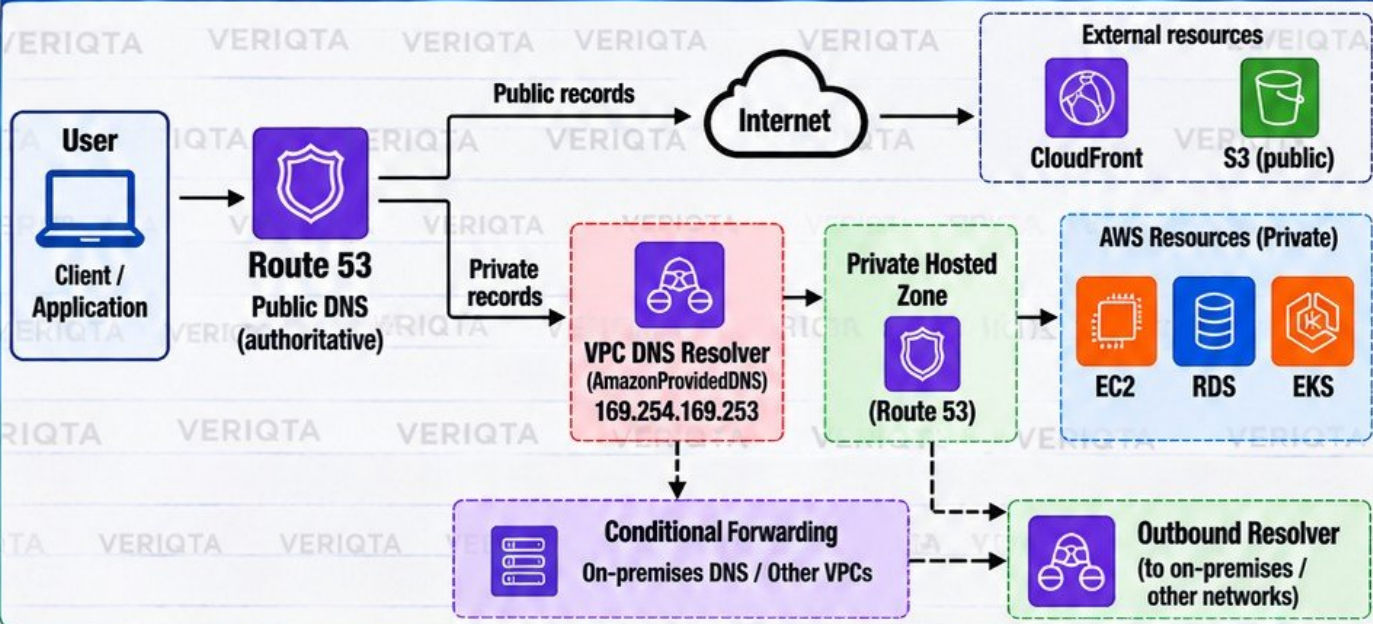


@veriqta

DNS Failure Inside AWS

Route 53, VPC DNS, private hosted zones, resolver behavior, TTLs, split-horizon DNS, and difficult resolution failures.

DNS ARCHITECTURE IN AWS



KEY COMPONENTS

	Route 53	Public DNS, hosted zones, routing policies, health checks.
	VPC DNS Resolver	AmazonProvidedDNS (169.254.169.253) for name resolution inside VPC.
	Private Hosted Zone	Private DNS for internal resources.
	Conditional Forwarding	Forward DNS queries to on-premises or other networks.
	Outbound Resolver	Send DNS queries from VPC to external DNS servers.
	Resolver Endpoints	Inbound and outbound endpoints for hybrid DNS.

COMMON DNS FAILURE SCENARIOS

- 1 Cannot resolve internal domain name.
- 2 Resolves to wrong IP (split-horizon conflict).
- 3 Intermittent resolution failures.
- 4 High latency in DNS responses.
- 5 NXDOMAIN for an existing record.
- 6 Stale records due to long TTL.
- 7 VPC cannot resolve public domains.
- 8 On-premises cannot resolve AWS private records.
- 9 Resolver endpoint misconfiguration.
- 10 DNS works from one subnet but not another.

KEY AREAS TO CHECK

- 1 Hosted zone (public or private).
- 2 DNS record type and value.
- 3 TTL and caching behavior.
- 4 VPC DNS settings (enableDnsSupport, enableDnsHostnames).
- 5 Route tables and resolver endpoints.
- 6 Security groups and NACLs.
- 7 Conditional forwarding rules.
- 8 Split-horizon DNS configuration.
- 9 Upstream DNS servers (on-premises).
- 10 Recent changes (records, VPC, networking).

TROUBLESHOOTING STEPS

- 1 Check if the domain should be public or private.
- 2 Test resolution from the affected instance.
- 3 Verify the correct hosted zone and record.
- 4 Check TTL and flush DNS cache.
- 5 Verify VPC DNS settings and route tables.
- 6 Test resolver endpoints and forwarding rules.
- 7 Check security groups, NACLs and network path.
- 8 Use VPC Flow Logs and Resolver query logs.

USEFUL COMMANDS

```
# Test DNS resolution
nslookup example.com
dig example.com

# Test against specific DNS server
dig @169.254.169.253 example.com

# Check DNS records
dig example.com A
dig example.com AAAA
dig example.com CNAME
dig example.com MX
```

KEY TAKEAWAYS

- ✓ VPC DNS (AmazonProvidedDNS) is required for name resolution inside VPC.
- ✓ Use private hosted zones for internal services.
- ✓ Be careful with split-horizon DNS to avoid conflicting records.
- ✓ Set appropriate TTLs to balance performance and change management.
- ✓ Use resolver endpoints for hybrid environments.
- ✓ Monitor DNS queries, latency and failure rates.
- ✓ Always verify recent changes when investigating DNS issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Load Balancers Under Pressure

ALB, NLB, connection behavior, health checks, deregistration, TLS, target failures, and misleading health signals.

AWS LOAD BALANCER TYPES

Application Load Balancer (ALB)



- ✓ Layer 7 (HTTP/HTTPS)
- ✓ Host and path-based routing
- ✓ Supports HTTP/2 and WebSockets
- ✓ Application-aware routing
- ✓ Best for web applications and microservices

Network Load Balancer (NLB)



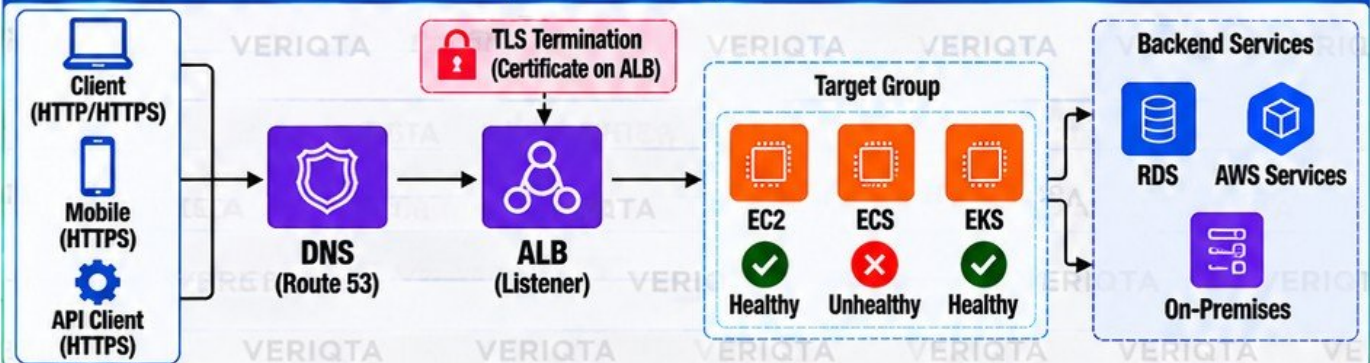
- ✓ Layer 4 (TCP/UDP/TLS)
- ✓ Ultra high performance
- ✓ Static IP and Elastic IP support
- ✓ Handles millions of connections
- ✓ Best for high-throughput, low-latency workloads

Gateway Load Balancer (GWLB)



- ✓ Layer 3 (IP)
- ✓ Transparent network gateway
- ✓ Used for virtual appliances (firewalls, IDS/IPS, etc.)
- ✓ Maintains original IP address
- ✓ Best for security and inspection tools

HOW A LOAD BALANCER WORKS (ALB EXAMPLE)



IMPORTANT COMPONENTS



Listener
Defines protocol (HTTP/HTTPS/TCP), port, and routing rules.



Target Group
Registers targets and performs health checks.



Health Check
Determines if targets are healthy or unhealthy.



Routing Rules (ALB)
Route based on host, path, headers, or query strings.



TLS/SSL
Terminate or pass through encryption.



Deregistration
Gracefully removes targets from service.

CONNECTION BEHAVIOR

- 1 Clients connect to the load balancer DNS name (Route 53).
- 2 ALB terminates TLS (or passes through).
- 3 Requests are routed to healthy targets based on rules.
- 4 Each request is a new connection (ALB) or can be reused (keep-alive).
- 5 NLB maintains long-lived TCP/UDP connections.
- 6 When a target is marked unhealthy, new requests stop, but existing connections may continue.

HEALTH CHECKS

- ✓ ALB: HTTP, HTTPS, or gRPC health checks at Layer 7.
- ✓ NLB: TCP or HTTP health checks at Layer 4.
- ✓ Configurable path, port, interval, timeout, and healthy/unhealthy thresholds.
- ✓ A target can be unhealthy but still respond to some requests.
- ✓ Health checks do not validate full application functionality.
- ✓ Misconfigured health checks can mark unhealthy targets as healthy (or vice versa).

COMMON FAILURE SCENARIOS

- 1 All targets show healthy but application returns errors.
- 2 Health checks pass but key functionality is broken.
- 3 High latency or connection timeouts.
- 4 Targets not deregistering properly (draining issues).
- 5 TLS certificate expired or misconfigured.
- 6 Incorrect security group or NACL blocking traffic.

DEREGISTRATION AND DRAINING

- 1 When a target is deregistered, new requests stop.
- 2 Existing connections continue until the deregistration delay (default 300s for ALB).
- 3 Adjust deregistration delay based on application behavior.
- 4 Ensure graceful shutdown to avoid dropped connections.

KEY TAKEAWAYS

- ✓ Choose the right load balancer for your workload.
- ✓ Configure meaningful health checks.
- ✓ Understand connection behavior and draining.
- ✓ Monitor target health, latency, and errors.
- ✓ Don't rely only on health checks as a measure of application health.
- ✓ Plan for failure and test failover scenarios.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



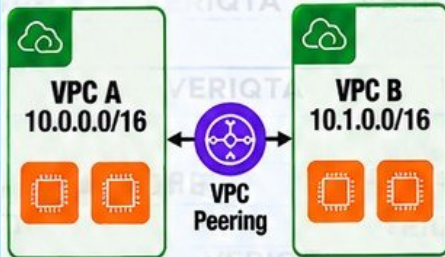
@veriqta

VPC Peering, Transit Gateway, and PrivateLink

Choosing connectivity based on routing scale, isolation, service exposure, operational ownership, and cost.

VPC PEERING

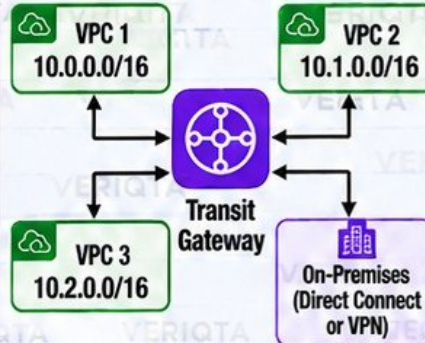
Direct, point-to-point connectivity between two VPCs.



- ✓ Simple and low latency.
- ✓ No transitive routing.
- ✓ Manual peering for each VPC.
- ✓ Best for few VPCs and simple use cases.
- ✓ You control route tables and security.

TRANSIT GATEWAY

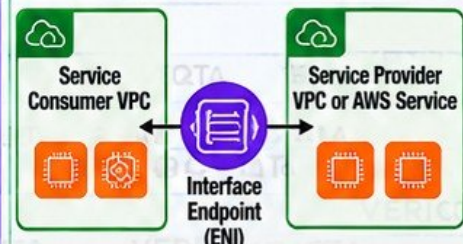
Hub-and-spoke connectivity for multiple VPCs and networks.



- ✓ Supports many VPCs and on-prem.
- ✓ Centralized routing (transitive).
- ✓ Simplifies management at scale.
- ✓ Supports route tables and segmentation.
- ✓ Best for large, multi-account environments.

PRIVATELINK

Securely expose and consume services privately over AWS network.



- ✓ Private access to services (no internet).
- ✓ Uses VPC endpoints (Interface Endpoints).
- ✓ Keep traffic within AWS network.
- ✓ Supports third-party and AWS services.
- ✓ Ideal for service exposure and isolation.

KEY COMPARISON

Feature	VPC Peering	Transit Gateway	PrivateLink
Connectivity Type	Point-to-point	Hub-and-spoke (many-to-many)	Service-based (consumer ↔ provider)
Transitive Routing	✗ Not supported	✓ Supported	✗ Not applicable
Scale	Few VPCs	Many VPCs and on-prem	Service-specific
Use Case	VPC to VPC	Multi-VPC, multi-account, on-prem	Expose or consume services privately
Isolation	Network level (VPC CIDR)	Centralized control and segmentation	Service-level isolation (endpoint)
Operational Ownership	Manual (per peering)	Centralized (TGW)	Consumer and provider managed
Cost	Low (data transfer applies)	Higher (TGW hourly + data transfer)	Higher (endpoint hourly + data transfer)
Best For	Simple, small-scale	Large, complex environments	Secure service access and exposure

ROUTING BEHAVIOR

- 1 VPC Peering requires route table updates on both VPCs.
- 2 Transit Gateway uses a central route table with transitive routing.
- 3 PrivateLink does not use your VPC CIDR for the service provider.
- 4 Return traffic follows the same path.
- 5 No transitive routing with VPC Peering.
- 6 Transit Gateway supports route tables per attachment.

COMMON USE CASES

- VPC Peering**
Connect two VPCs (e.g. dev and prod).
- Transit Gateway**
Connect multiple VPCs, multiple accounts, and on-premises networks.
- PrivateLink**
Access AWS services (S3, DynamoDB, etc) privately or expose your own services to other VPCs or customers.

KEY TAKEAWAYS

- ✓ Choose VPC Peering for simple, low-scale VPC-to-VPC connectivity.
- ✓ Choose Transit Gateway for large, multi-VPC and hybrid environments.
- ✓ Choose PrivateLink for secure service exposure and consumption.
- ✓ Understand routing behavior, isolation, operational ownership, and cost before deciding.
- ✓ Use proper segmentation, route tables, and security controls.
- ✓ Avoid using VPC Peering for transitive routing.
- ✓ Monitor connectivity, route propagation, and data transfer costs.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

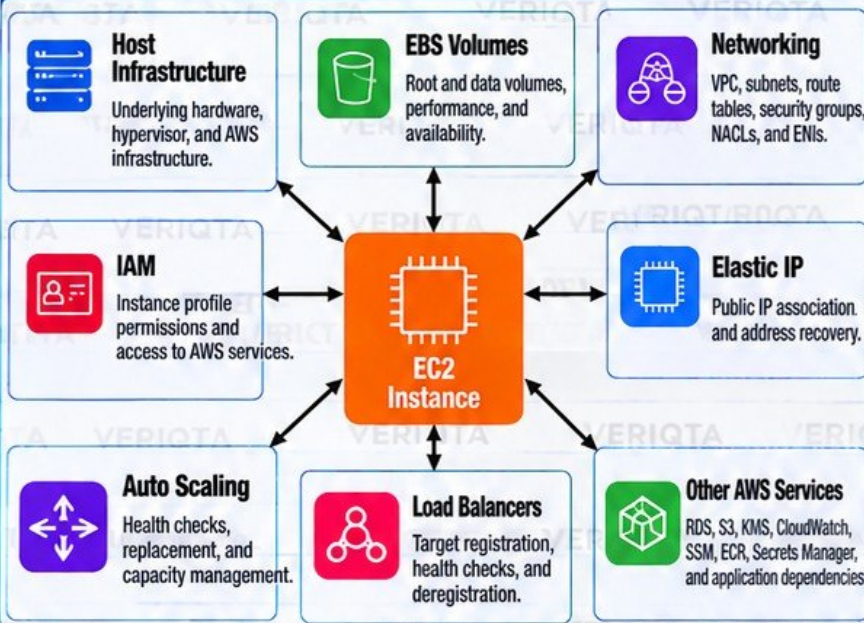


@veriqta

EC2 Failure Is More Than Instance Failure

Host problems, EBS dependencies, networking, status checks, Auto Scaling, replacement, and recovery decisions.

EC2 AND ITS DEPENDENCIES



COMMON FAILURE AREAS

- Instance issues**
Unresponsive, high CPU/memory, kernel panic, or OS corruption.
- Host problems**
Underlying hardware or hypervisor failure (reboot, stop, or terminate).
- EBS dependencies**
Volume failure, I/O issues, detached volume, or performance limits.
- Networking**
ENI issues, security group changes, NACL blocks, route table misconfiguration.
- AWS service dependencies**
RDS, S3, KMS, Secrets Manager, etc.
- Auto Scaling**
Unhealthy instances, failed replacements, scaling misconfiguration.

EC2 STATUS CHECKS

System Status Check



- Checks the underlying AWS infrastructure (host).
- If it fails, it's usually not your instance.
- You cannot fix it — replace the instance.
- View in EC2 console or CloudWatch.

Instance Status Check



- Checks the OS, networking, and instance configuration.
- Failure can be due to OS crashes, networking issues, or configuration problems.
- You can troubleshoot and fix the issue.
- View in EC2 console or CloudWatch.

AUTO SCALING BEHAVIOR



- Uses health checks (EC2 or ELB).
- Replaces unhealthy instances automatically.
- Can terminate and launch new instances in different AZs.
- Requires correct Launch Template or AMI.
- Health check grace period prevents premature termination.
- Scaling policies control desired capacity.

RECOVERY DECISIONS



- Recover the instance**
Troubleshoot and fix the underlying issue.
- Stop and start**
Use for instance or OS issues (EBS persists).
- Reboot**
For temporary issues (faster but may not fix all).
- Replace the instance**
Required for host failures or persistent issues.
- Restore from backup**
Use AMI or EBS snapshots.
- Check and restore dependencies**
Verify EBS, networking, IAM, and connected services.

COMMON FAILURE SCENARIOS

- Instance status check failed due to OS crash.
- System status check failed (host hardware issue).
- EC2 cannot reach EBS volume.
- Instance lost network connectivity (SG, NACL, route table).
- Auto Scaling keeps terminating new instances.
- Instance is healthy but fails application health checks.
- Elastic IP not reassociated after instance replacement.
- Dependencies (RDS, S3, KMS, etc.) are unreachable.

USEFUL TOOLS AND LOGS



- EC2 Console**
Status checks, instance actions.
- CloudWatch**
Metrics, logs, and alarms.
- VPC Flow Logs**
Network traffic analysis.
- SSM Session Manager**
Connect and troubleshoot.
- EBS Snapshots**
Recover data if instance is lost.

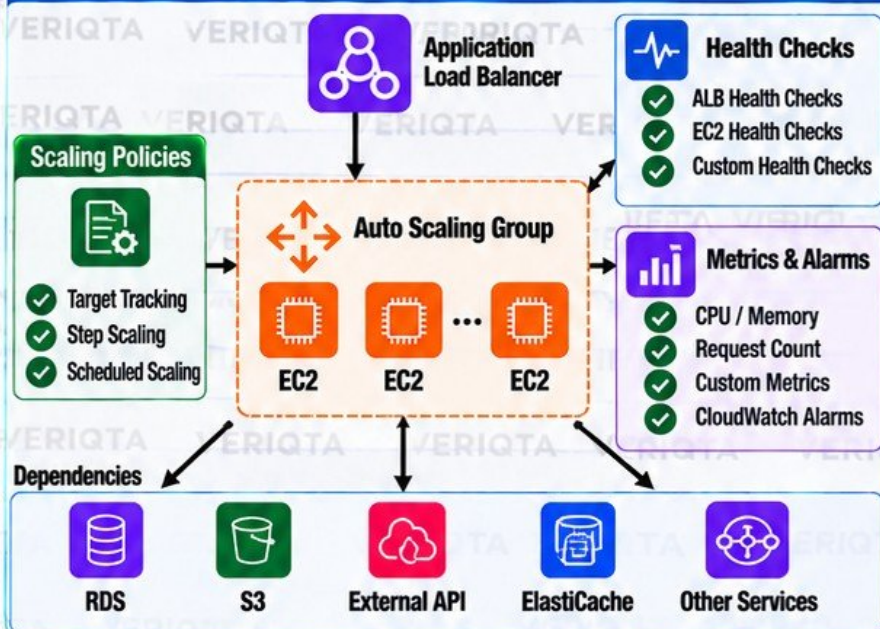
KEY TAKEAWAYS

- EC2 failures can be caused by more than the instance itself.
- Always check both system and instance status checks.
- Understand and monitor dependencies (EBS, networking, IAM, other services).
- Use Auto Scaling with proper health checks and Launch Templates.
- Have backup and recovery strategies (AMI, EBS snapshots).
- Choose the right recovery action based on the root cause.

Auto Scaling Can Amplify an Incident

Bad health checks, dependency failures, scaling storms, cold starts, quotas, and runaway replacement behavior.

AUTO SCALING ARCHITECTURE



SCALING FLOW DURING AN INCIDENT

- A dependency starts failing (e.g. database, API, or cache).
- Health checks fail or latency increases.
- Auto Scaling terminates unhealthy instances.
- New instances are launched to replace them.
- New instances also fail (same dependency issue).
- Scaling continues to launch and terminate instances.
- This creates a scaling storm amplifying the incident.

COMMON CAUSES

- Bad Health Checks**
Health check endpoint depends on a failing service.
- Dependency Failures**
Database, API, cache or downstream service is down.
- Scaling Storms**
ASG keeps launching and terminating instances.
- Cold Starts**
New instances take time to initialize and become healthy.
- Service Quotas**
Hit EC2, ENI, or other limits during rapid scaling.
- Runaway Replacement**
Unhealthy instances keep getting replaced without fixing the root cause.

EXAMPLE: SCALING STORM



IMPACT

- Increased costs**
Many unnecessary instances are launched.
- Service instability**
Continuous termination and launch cycles.
- Resource exhaustion**
EC2, ENI, and other service quotas can be reached.
- Longer recovery time**
New instances cannot become healthy.
- Additional load**
Can worsen the failing dependency.
- Operational noise**
More alarms and incident fatigue.

PREVENTION AND BEST PRACTICES

- Use robust health checks that do not depend on external services.
- Set appropriate health check grace periods.
- Choose the right scaling policies and cool-down periods.
- Use dependency-aware health checks.
- Set sensible minimum, maximum and desired capacity.
- Monitor and set service quotas (EC2, ENI, etc.).
- Test failure scenarios regularly.
- Use multi-AZ and resilient architecture for dependencies.

USEFUL AWS TOOLS

- EC2 Auto Scaling**
Scaling policies, health checks, and activity logs.
- Application Load Balancer**
Target group health checks and access logs.
- Amazon CloudWatch**
Metrics, alarms, and logs.
- AWS Service Quotas**
Monitor and request quota increases.
- AWS Systems Manager**
Automation and recovery actions.

KEY TAKEAWAYS

- Auto Scaling can amplify an incident instead of fixing it.
- Health checks must be designed carefully.
- Understand dependencies and failure impact.
- Use proper grace periods and scaling limits.
- Monitor quotas to avoid hitting limits.
- Test failure scenarios and scaling behavior.
- Have a clear recovery plan for scaling storms.
- Always investigate the root cause, not just increase capacity.

ECS vs EKS vs Lambda

Compare operational control, abstraction, scaling characteristics, failure surfaces, platform ownership, and workload fit.

Amazon ECS

Container orchestration service (managed by AWS).

- Runs containerized applications using task definitions.
- Simple and fully managed control plane.
- Integrates tightly with other AWS services (ALB, IAM, CloudWatch, etc).
- Supports EC2 and Fargate launch types.
- Best for teams that want containers without managing Kubernetes.

USE CASES


Web applications


Microservices


Batch processing


APIs and backend services


Containerized workers


Enterprise applications

Amazon EKS

Managed Kubernetes service (operated by AWS).

- Runs Kubernetes on AWS.
- Full Kubernetes control and flexibility.
- AWS manages the control plane, you manage the worker nodes (or use Fargate).
- Ideal for complex, large-scale and multi-environment deployments.
- Best for teams that need Kubernetes ecosystem, advanced networking, and customization.

USE CASES


Large-scale microservices


Multi-team platforms


Complex workloads


Hybrid and multi-cloud


CI/CD platforms


Stateful applications

AWS Lambda

Serverless compute service (event-driven).

- Runs code without managing servers.
- Automatically scales from zero to millions.
- You only pay for the execution time.
- No infrastructure management.
- Best for event-driven, short-running, and stateless workloads.

USE CASES


Event-driven applications


APIs (API Gateway)


Data processing


Real-time file processing


Automation and integration


Scheduled tasks

KEY COMPARISON

Feature	ECS	EKS	Lambda
 Abstraction Level	Container orchestration (AWS-native)	Kubernetes (standard)	Serverless (function-level)
 Operational Control	Lower (managed by AWS)	Higher (more control and customization)	Minimal (fully managed)
 Scaling Characteristics	Manual, scheduled or automatic (service and task scaling)	Highly scalable (HPA, Cluster Autoscaler)	Automatic (scales from zero)
 Failure Surface	Service, task, host or Fargate issues	Control plane, nodes, pods, networking, etc.	Function, dependency or service limits
 Platform Ownership	AWS manages ECS control plane	AWS manages control plane, you manage nodes and workloads	AWS manages everything
 Compute Model	Containers (EC2 or Fargate)	Containers (EC2 or Fargate)	Functions (event-driven)
 Best For	Simple to complex containerized applications	Large, complex, Kubernetes-native workloads	Event-driven, stateless, short-running tasks
 Cost Model	Pay for compute (EC2/Fargate) and usage	Pay for worker nodes (or Fargate) + control plane	Pay per invocation and execution time

COMMON FAILURE SCENARIOS

- ECS tasks failing health checks.
- ECS service unable to place new tasks (insufficient capacity).
- EKS node or pod failures.
- EKS control plane or networking issues.
- Lambda function timeouts.
- Lambda hitting concurrency or service limits.
- Downstream dependency failures (DB, API, etc).
- Misconfiguration, IAM, or resource quotas.

THINGS TO CONSIDER

- Workload type (long-running vs short-lived).
- Operational complexity and team expertise.
- Scaling requirements and traffic patterns.
- Failure impact and recovery strategy.
- Integration with existing AWS services.
- Cost and resource efficiency.
- Networking and security requirements.
- Future growth and multi-environment needs.

KEY TAKEAWAYS

- ECS is ideal for simple container management with low operational overhead.
- EKS provides full Kubernetes capabilities for complex and large-scale workloads.
- Lambda is best for event-driven, stateless, and short-lived tasks.
- Choose the right service based on workload fit, team expertise, operational control, and cost.
- Understand the failure surfaces and plan monitoring, alerting, and recovery.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

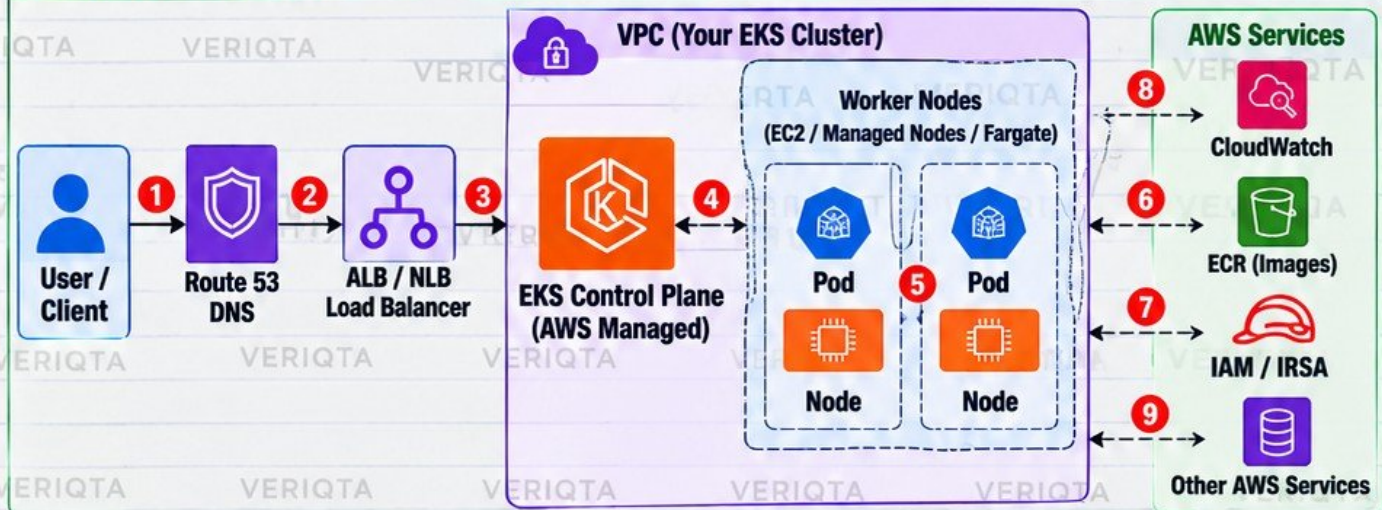


@veriqta

EKS From Request to Running Pod

AWS infrastructure, Kubernetes scheduling, VPC CNI, IAM, load balancing, DNS, and the dependencies behind one workload.

1. END-TO-END FLOW



1 Client Request

User sends a request to your application domain.

2 DNS Resolution (Route 53)

Route 53 resolves the domain to the ALB or NLB endpoint.

3 Load Balancer

ALB/NLB receives the request and forwards it to a healthy pod target (usually a Service).

4 EKS Control Plane

The control plane (managed by AWS) handles API requests, scheduling decisions, and cluster state.

5 Pod Scheduling

The scheduler places the pod on a suitable node based on resources, constraints, and policies.

6 Pull Container Image

The node pulls the image from ECR using IAM permissions.

7 Pod Identity (IAM / IRSA)

Pods access AWS services using IAM roles for service accounts (IRSA), not hardcoded credentials.

8 Logging and Monitoring

Pods, nodes, and cluster components send logs and metrics to CloudWatch (and other observability tools).

9 Access to Other AWS Services

The running pod accesses other AWS services (RDS, S3, DynamoDB, SQS, etc.) through VPC and IAM.

KEY COMPONENTS INVOLVED

-  **EKS Control Plane** – Kubernetes API, etcd, scheduler (managed by AWS)
-  **Worker Nodes** – EC2, managed node groups or Fargate
-  **Load Balancer** – ALB or NLB
-  **Route 53** – DNS and health checks
-  **VPC** – subnets, route tables, security groups
-  **VPC CNI** – assigns pod IPs from VPC subnets
-  **IAM / IRSA** – pod identity and permissions
-  **CloudWatch** – logs, metrics, alarms
-  **ECR** – container images
-  **Other AWS Services** – RDS, S3, DynamoDB, SQS, etc.

VPC CNI AND POD NETWORKING

- Each pod gets a VPC IP address.
- IPs are assigned from the subnet using the VPC CNI.
- Security groups can be applied to pods (security groups for pods).
- Pod-to-pod and pod-to-service traffic stays inside the VPC.
- Network policies (Calico/Cilium) can control pod traffic.
- Node ENI and IP limits can impact pod density.

COMMON FAILURE POINTS

- ❗ DNS misconfiguration or wrong target.
- ❗ ALB health checks failing.
- ❗ Image pull failures from ECR.
- ❗ IAM/IRSA permissions issues.
- ❗ Insufficient pod IPs in subnet (VPC CNI).
- ❗ Node capacity or instance limits.
- ❗ Pod stuck in Pending or CrashLoopBackOff.
- ❗ Network policies blocking traffic.
- ❗ Security group or NACL misconfiguration.
- ❗ Upstream AWS service throttling or failure.

SENIOR ENGINEER TIP

A pod running does not mean the application is healthy. Always verify end-to-end: DNS → Load Balancer → Service → Pod → Dependencies.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

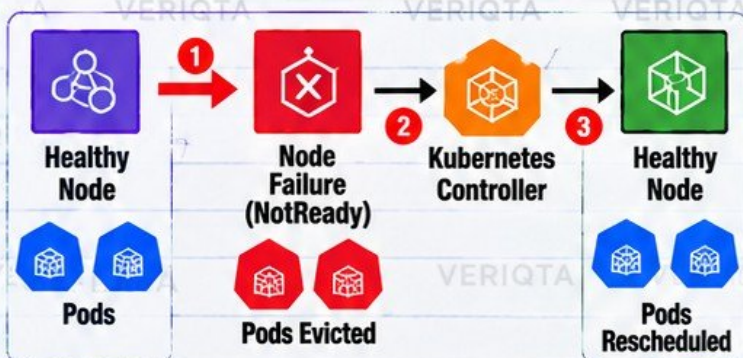


@veriqta

EKS Node Failure

Node conditions, pod eviction, rescheduling, PDBs, topology, capacity availability, and recovery behavior.

1. EKS NODE FAILURE FLOW



- 1 Node becomes NotReady due to failure (hardware, network, OS, etc).
- 2 Kubernetes detects the failure and evicts pods (following PDB rules).
- 3 Pods are rescheduled to other healthy nodes based on resources, constraints, and topology settings.

2. COMMON NODE CONDITIONS

Condition	Meaning	Common Causes
Ready False	Node is not ready to run pods.	Network issues, kubelet down, system failure.
MemoryPressure True	Node memory is low.	High memory usage, memory leaks, large pods.
DiskPressure True	Node disk is low.	Full disk, container logs, image cleanup needed.
PIDPressure True	Too many processes.	PID exhaustion, runaway processes.
NetworkUnavailable True	Node network not ready.	CNI issues, ENI limits, network misconfiguration.
Ready Unknown	Node status unknown.	Lost communication with node (possible crash).

3. WHY NODES FAIL



Instance failure



Underlying host issue



Out of CPU or memory



Disk full



CNI / network issues



AZ or infrastructure issue



Manual maintenance or node drain

4. POD EVICTION BEHAVIOR

- Pods are evicted when a node becomes NotReady.
- Eviction respects Pod Disruption Budgets (PDBs).
- If a PDB blocks eviction, pods may stay in Terminating state until the node is forcefully removed.
- Stateful workloads may take longer to reschedule.
- DaemonSet pods are recreated on new nodes automatically.

5. RESCHEDULING FACTORS

- Available node capacity (CPU, memory, GPU).
- Node selectors and node affinity rules.
- Taints and tolerations.
- Topology spreading constraints.
- Availability Zone and capacity type (On-Demand or Spot).
- PDB restrictions.
- Cluster Autoscaler (if enabled) may provision new nodes if there is no capacity.

6. USEFUL COMMANDS

<code>kubectl get nodes -o wide</code>	Check node status and details
<code>kubectl describe node <node-name></code>	View node conditions and events
<code>kubectl get pods -o wide</code>	See where pods are running
<code>kubectl get events --sort-by=.lastTimestamp</code>	Check recent events
<code>kubectl describe pod <pod-name></code>	View eviction or scheduling reason
<code>kubectl get pdb</code>	Check Pod Disruption Budgets
<code>kubectl get nodes grep -i NotReady</code>	Find unhealthy nodes
<code>kubectl drain <node-name> --ignore-daemonsets</code>	Manually drain a node (maintenance)
<code>kubectl uncordon <node-name></code>	Bring a node back after maintenance

7. RECOVERY AND RESILIENCE

- 1 Identify the root cause (instance, network, disk, etc).
- 2 Replace the node (Auto Scaling or managed node group).
- 3 Verify pods are rescheduled and applications are healthy.
- 4 Ensure PDBs are set appropriately for critical workloads.
- 5 Use multi-AZ node groups for higher availability.
- 6 Monitor node health (CloudWatch, node exporter, alerts).

8. KEY TAKEAWAYS

- Node failures are normal in production.
- Design with multiple AZs and enough capacity.
- Use PDBs to control disruption.
- Understand your workload topology and constraints.
- Monitor node conditions and act before impact.
- Automate recovery using managed node groups and autoscaling.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

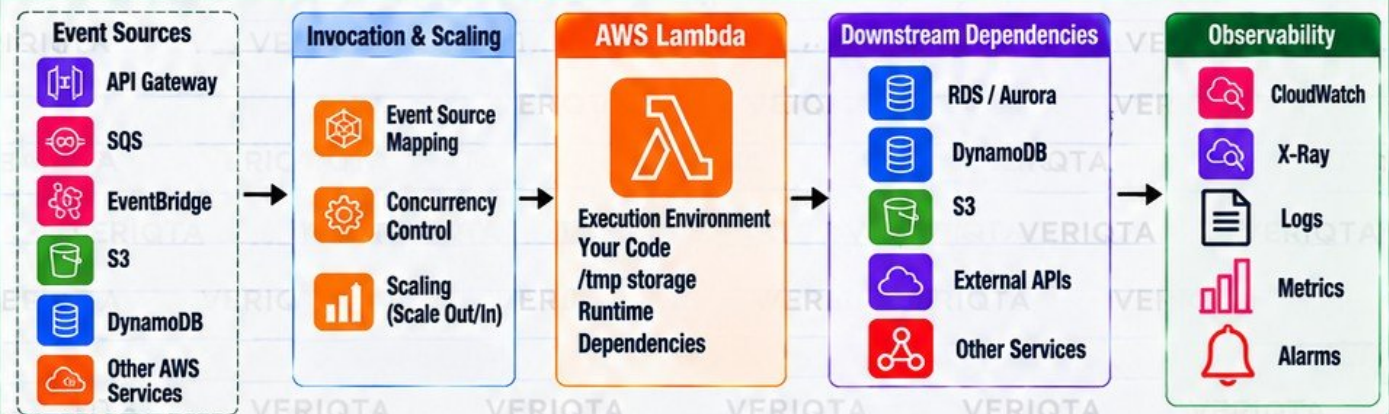


@veriqta

Serverless Under Real Load

Lambda concurrency, throttling, cold starts, downstream pressure, retries, idempotency, and event-source behavior.

1. REQUEST FLOW AND KEY COMPONENTS



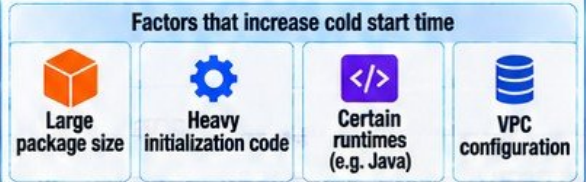
2. CONCURRENCY AND SCALING

- ✓ Each function has a regional concurrency limit (default 1,000).
- ✓ Lambda automatically scales based on incoming requests.
- ✓ You can set reserved concurrency to protect critical functions.
- ✓ Account limit can throttle all functions if reached.



3. COLD STARTS

- ✓ A cold start happens when a new execution environment is created.
- ✓ More common with infrequent traffic.
- ✓ Affected by runtime, package size and initialization code.
- ✓ Provisioned Concurrency reduces cold starts for latency-sensitive workloads.



4. THROTTLING AND ITS IMPACT

- ✓ Throttling occurs when concurrency limits are exceeded.
- ✓ Returns 429 TooManyRequestsException.
- ✓ Can cause retries and increased load on upstream systems.
- ✓ Monitor throttles to detect scaling or design issues.

5. EVENT-SOURCE BEHAVIOR

- SQS: retries with backoff, moves to DLQ after max attempts.
- EventBridge: retries for 24 hours with exponential backoff.
- S3: at-least-once delivery, may deliver duplicate events.
- API Gateway: no automatic retries. Client must retry.

6. RETRIES AND IDEMPOTENCY

- ❗ Retries can cause duplicate invocations.
- ❗ Design your functions to be idempotent.
- ✓ Use unique request IDs or conditional writes.
- ❗ Handle partial failures gracefully.
- ✓ Be aware of retry behavior from different event sources.

7. REAL-WORLD FAILURE SCENARIOS

Scenario	Symptoms	Likely Cause	Impact
High traffic spike	429 errors	Concurrency limit reached	Requests dropped or retried
Increased latency	Slow responses	Cold starts	User-facing delays
Downstream DB slow	Function timeouts	RDS/DynamoDB latency	Retries, more load
SQS backlog growing	Messages not processed	Function errors or throttling	Increased message age
Third-party API slow	Function timeouts	External API latency	Retries and cascading failures

8. USEFUL COMMANDS (AWS CLI)

<code>aws lambda get-function --function-name <name></code>	View configuration
<code>aws lambda get-metric-statistics --metric-name Throttles --namespace AWS/Lambda</code>	Check throttles
<code>aws lambda list-functions</code>	List functions
<code>aws lambda put-provisioned-concurrency-config --function-name <name> --provisioned-concurrent-executions 10</code>	Enable provisioned concurrency

9. KEY TAKEAWAYS

- ✓ Understand your concurrency limits.
- ✓ Design for failures and retries.
- ✓ Make your functions idempotent.
- ✓ Use reserved or provisioned concurrency for critical workloads.
- ✓ Monitor cold starts, throttles, errors and downstream latency.
- ✓ Test with realistic load patterns before production.

10. SENIOR ENGINEER TIPS

- ✓ Use asynchronous architecture (SQS, EventBridge) to isolate downstream dependencies.
- ✓ Set reserved concurrency for critical and non-critical functions.
- ✓ Keep package size small and initialize efficiently.
- ✓ Use DLQs and alarm on message age and failure rates.
- ✓ Monitor downstream services and set appropriate timeouts.
- ✓ Review event source retry policies and adjust application logic.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

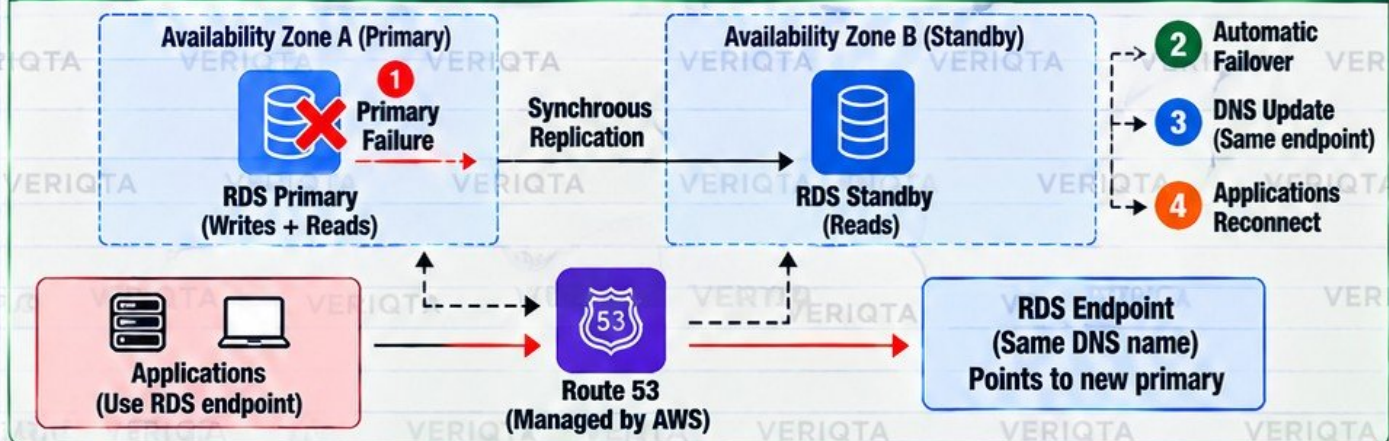


@veriqta

RDS Failure and Recovery

Primary failure, Multi-AZ behavior, failover, DNS changes, connection recovery, transactions, and application consequences.

1. MULTI-AZ ARCHITECTURE AND FAILOVER FLOW



2. FAILOVER STEPS (WHAT HAPPENS)

- 1 Primary instance becomes unavailable (due to hardware, network, OS, etc.).
- 2 AWS promotes the standby to primary.
- 3 RDS endpoint DNS is updated to the new primary.
- 4 Existing connections are dropped.
- 5 Applications reconnect to the same endpoint.
- 6 New primary begins handling read and write traffic.

3. DNS AND CONNECTION BEHAVIOR

- 1 RDS uses a DNS name (CNAME) for the endpoint.
- 2 During failover, DNS is updated to the new primary IP.
- 3 DNS TTL is typically 5 seconds (cannot be modified).
- 4 Existing TCP connections are terminated.
- 5 Applications must handle reconnection and retries.
- 6 Long-lived connections and connection pools can break.

4. APPLICATION IMPACT

- ✓ Active connections are dropped (you get errors).
- ✓ In-flight transactions are rolled back.
- ✓ Read and write operations fail during the failover window (usually 30-120 seconds).
- ✓ Connection pools must refresh connections.
- ✓ Applications should implement retry with backoff.
- ✓ User-facing errors may occur if retries are not handled.

6. RECOVERY AND BEST PRACTICES

- ✓ Use Multi-AZ for production databases.
- ✓ Implement retry with exponential backoff in applications.
- ✓ Use short connection timeouts.
- ✓ Make your application stateless and handle transient failures.
- ✓ Monitor RDS events, CloudWatch metrics and alarms.
- ✓ Regularly test failover (e.g. reboot primary).
- ✓ Use RDS Proxy to manage connections and improve resilience.

5. RDS FAILURE SCENARIOS

Scenario	What Happens	Impact
Instance hardware failure	Automatic failover to standby	Short downtime, connections dropped
AZ outage	Failover to other AZ	Applications must reconnect
Storage failure	Failover to standby with data from replication	Possible data loss if replication lagged
Network failure	Failover triggered by health checks	Short outage, connections drop
Human error (reboot/delete)	Failover if Multi-AZ enabled	Downtime if single-AZ

7. USEFUL COMMANDS (AWS CLI)

<code>aws rds describe-db-instances</code>	View instance status and role (primary/standby)
<code>aws rds describe-events</code>	Check recent failure or failover events
<code>aws rds describe-db-instances --query "DBInstances[*].[DBInstanceIdentifier, DBInstanceStatus, AvailabilityZone]"</code>	Quick view of instance status and AZ
<code>aws rds describe-db-clusters</code>	(Aurora) view cluster and failover status
<code>aws cloudwatch get-metric-statistics</code>	Check RDS metrics (CPU, connections, etc.)

8. KEY TAKEAWAYS

- ✓ Multi-AZ improves availability but does not eliminate downtime.
- ✓ Failover changes the underlying IP but the endpoint remains the same.
- ✓ Applications must handle dropped connections and retries.
- ✓ Test failover and monitor to ensure recovery works as expected.

9. COMMON MISTAKES

- ✗ Relying on long-lived database connections.
- ✗ Not handling connection errors and retries.
- ✗ Using single-AZ for critical workloads.
- ✗ Ignoring transaction rollbacks during failover.
- ✗ Not testing failover in a non-production environment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

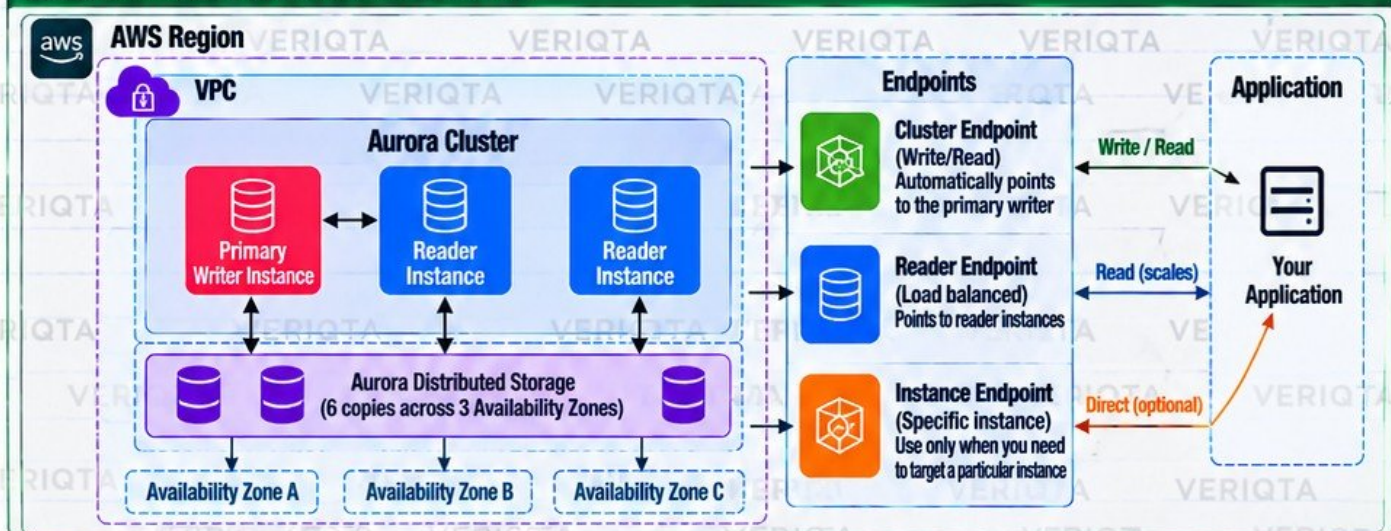


@veriqta

Aurora Is Distributed, Your Application Still Matters

Cluster architecture, replicas, failover, endpoints, connection management, and application-side resilience.

1. AURORA CLUSTER ARCHITECTURE



2. HOW AURORA HANDLES FAILOVER

- 1 Primary instance fails (hardware, network, OS, etc).
- 2 Aurora automatically promotes a healthy reader to primary.
- 3 Cluster endpoint is updated to point to the new primary (usually within 30–60 seconds).
- 4 Existing connections to the old primary are dropped.
- 5 Applications must reconnect using the cluster endpoint.
- 6 Uncommitted transactions are rolled back.

3. ENDPOINT BEHAVIOR DURING FAILOVER

Endpoint Type	Points To	Changes During Failover	Use Case
Cluster Endpoint	Primary writer instance	Automatically updated to new primary	Application read/write connections
Reader Endpoint	Reader instances (load balanced)	Continues to route to available readers	Read-only workloads
Instance Endpoint	Specific instance	Does not change (still points to same instance ID)	Admin tasks, troubleshooting

4. REPLICAS AND READ SCALING

- ✓ Aurora uses storage-level replication across multiple AZs (6 copies of data).
- ✓ Reader instances provide horizontal read scaling.
- ✓ You can have multiple readers (up to 15, depending on version).
- ✓ Readers can be in the same region or cross-region.
- ✓ Each reader has its own endpoint.

5. CONNECTION MANAGEMENT

- ✓ Use the cluster endpoint, not instance endpoints, in applications.
- ✓ Implement automatic reconnection with exponential backoff.
- ✓ Set appropriate connection timeouts (do not wait indefinitely).
- ✓ Use connection pooling (e.g., RDS Proxy, PgBouncer, HikariCP).
- ✓ Handle dropped connections and retry at the application layer.
- ✓ Monitor and set max_connections based on workload.

6. APPLICATION-SIDE RESILIENCE

- ✓ Always use the cluster endpoint for write operations.
- ✓ Implement retry logic for transient connection errors.
- ✓ Handle transaction rollbacks gracefully.
- ✓ Design for idempotent writes to avoid duplicate data.
- ✓ Use read replicas for read-heavy workloads.
- ✓ Monitor application errors and database metrics.

7. COMMON FAILURE SCENARIOS

Scenario	What Happens	Impact	How to Handle
Primary instance failure	Automatic failover to reader	Short downtime (30–60 seconds)	View cluster endpoint and retry logic
AZ outage	Failover to another AZ	Minimal downtime	No action needed (if Multi-AZ enabled)
Storage node failure	Automatically repaired using 6 copies	No downtime	Handled by Aurora
Network issue	Connections dropped	Application errors	Retry with backoff
Reader instance failure	Removed from reader endpoint	Reduced read capacity	Monitor and replace automatically

8. KEY TAKEAWAYS

- ✓ Aurora is distributed storage, not just a single database.
- ✓ Failover is automatic, but applications must handle reconnections.
- ✓ Use the right endpoint for the right workload.
- ✓ Connection management and retry logic are critical.
- ✓ Design your application to be resilient to transient failures.

9. USEFUL COMMANDS (AWS CLI)

<code>aws rds describe-db-clusters</code>	View Aurora cluster details
<code>aws rds describe-db-instances</code>	List cluster instances (writer/reader)
<code>aws rds describe-db-cluster-endpoints</code>	View endpoints
<code>aws rds failover-db-cluster --db-cluster-identifier <id></code>	Manually trigger failover (test)
<code>aws rds describe-events --source-type db-cluster</code>	Check cluster events
<code>aws rds describe-db-cluster-parameter-groups</code>	View parameter groups



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

DynamoDB Hot Partitions and Throttling

Partition-key design, access patterns, capacity, adaptive behavior, throttling evidence, and architectural correction.

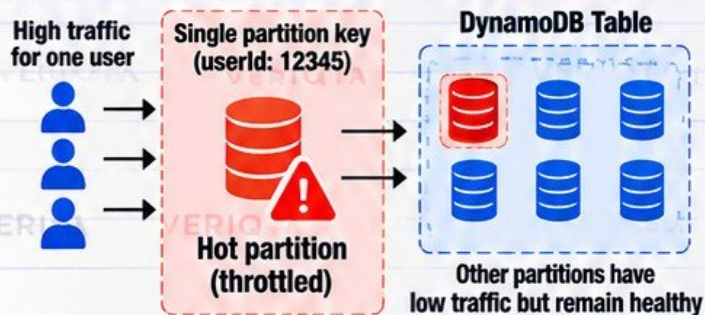
WHAT IS A HOT PARTITION?

- A hot partition receives a disproportionate amount of traffic for a single partition key.
- DynamoDB scales partitions automatically, but each partition has limits.
- If one partition key gets too much traffic, it can be throttled even if your table has high overall capacity.
- Hot partitions lead to throttling, increased latency and failed requests.

WHY HOT PARTITIONS HAPPEN

- Poor partition key design (e.g. a single user ID gets high traffic).
- Uneven access patterns (e.g. viral content, flash sales, hotspot tenants).
- Time-based keys without distribution (e.g. yyyy-mm-dd).
- Write-heavy workloads to the same key.
- Sudden traffic spikes (e.g. game events).
- Small key space or predictable keys.

EXAMPLE: HOT PARTITION SCENARIO



PARTITION KEY DESIGN PATTERNS

- 1 Use high-cardinality keys (e.g. user ID, device ID).
- 2 Avoid monotonically increasing keys.
- 3 Distribute time-based keys (add a random suffix).
- 4 Use composite keys when needed (e.g. tenantId#userId).
- 5 Consider write sharding for high-write workloads.
- 6 Design for unpredictable traffic spikes.

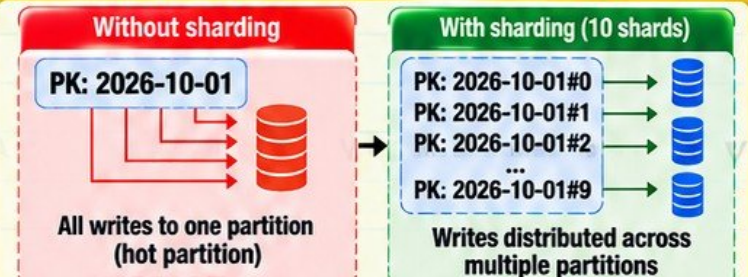
TIME-BASED KEY WITH SHARDING

Partition key format:

yyyy-mm-dd#shard-id

- Adds a random or hash-based suffix to spread load.
- Increases partition key cardinality.
- Helps avoid concentrated traffic on a single partition.

EXAMPLE: WRITE SHARDING



THROTTLING EVIDENCE

- ProvisionedThroughputExceededException or ThrottlingException.
- Increased latency and retry attempts.
- High ConsumedWriteCapacityUnits for a small number of partition keys.
- CloudWatch metric: ThrottledRequests.
- DynamoDB Contributor Insights showing top keys.
- Application errors or slow responses.

USEFUL COMMANDS (AWS CLI)

```
# View throttled requests
aws cloudwatch get-metric-statistics \
  --namespace AWS/DynamoDB \
  --metric-name ThrottledRequests \
  --dimensions Name=TableName,Value=Orders \
  --start-time 2026-10-01T00:00:00Z \
  --end-time 2026-10-02T00:00:00Z \
  --period 300 --statistics Sum
```

CONTRIBUTOR INSIGHTS

```
aws dynamodb describe-contributor-
insights \
  --table-name Orders \
  --query \
  'ContributorInsights[].[
    Status:Status,
    Rule:ContributorInsightsRule.RuleName]'
```

ARCHITECTURAL CORRECTIONS

- 1 Redesign partition key for higher cardinality.
- 2 Use write sharding or random suffix.
- 3 Implement adaptive capacity across more keys.
- 4 Split workloads by tenant, region or use composite keys.
- 5 Add caching for read-heavy hotspots (e.g. DAX or ElastiCache).
- 6 Monitor with CloudWatch and Contributor Insights.

KEY TAKEAWAYS

- ✓ A hot partition can throttle your application even if the table has high overall capacity.
- ✓ Partition key design is the most important decision in DynamoDB.
- ✓ Use sharding or composite keys for high-write or unpredictable traffic.
- ✓ Always monitor throttling metrics and top keys.
- ✓ Design for scale and traffic spikes from the beginning.

S3 Is Simple Until It Becomes Critical Infrastructure

Permissions, encryption, lifecycle, versioning, replication, event dependencies, request patterns, and recovery.

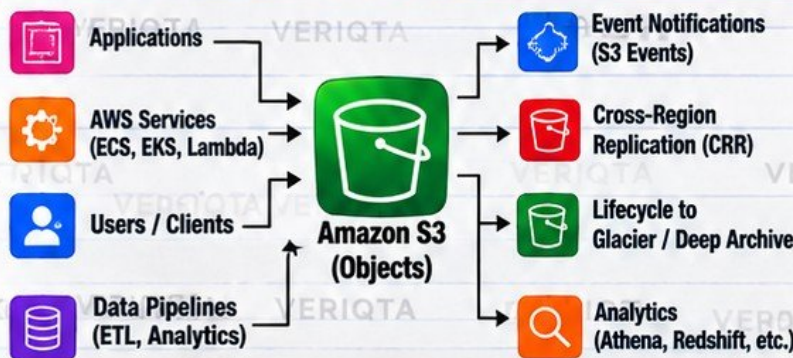
WHY S3 BECOMES CRITICAL

- 1 Stores important business data.
- 2 Used by many services and applications.
- 3 Integrated with analytics, backups, media, logs, websites and more.
- 4 A single misconfiguration can expose data or break multiple systems.
- 5 Deletions, overwrites or permission changes can be hard to recover from.

COMMON S3 USE CASES



S3 ARCHITECTURE AND INTEGRATIONS



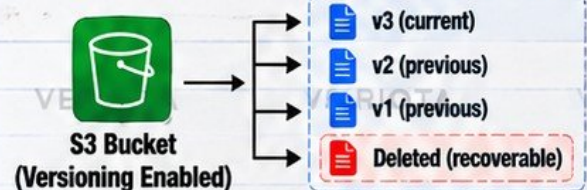
KEY S3 FEATURES

- 1 Permissions – control access with IAM, bucket policies and access points.
- 2 Encryption – SSE-S3, SSE-KMS or client-side.
- 3 Lifecycle – move objects to cheaper storage.
- 4 Versioning – protect against accidental deletions and overwrites.
- 5 Replication – cross-region or same-region.
- 6 Event Notifications – trigger Lambda, SNS, SQS.
- 7 Storage Classes – Standard, IA, One Zone-IA, Glacier, Deep Archive.
- 8 Data Access Patterns – design for your workload.
- 9 Recovery – restore deleted or previous versions.

S3 STORAGE CLASSES (WHEN TO USE)

Storage Class	Use Case	Key Point
Standard	Frequently accessed	High durability, low latency
Intelligent-Tiering	Unknown or changing	Automatically moves objects
Standard-IA	Infrequent access	Lower cost, retrieval fee
One Zone-IA	Non-critical, single AZ	Lower cost, no multi-AZ
Glacier Instant	Rarely accessed, fast restore	Low cost, millisecond access
Glacier Flexible	Long-term archive	Minutes to hours retrieval
Glacier Deep Archive	Very long-term	Lowest cost, hours to restore

VERSIONING AND RECOVERY



- ✓ Keeps multiple versions of an object.
- ✓ Protects against accidental deletions and overwrites.
- ✓ You can restore any previous version.
- ✓ Supports MFA Delete for stronger protection.

REPLICATION OPTIONS

Same-Region Replication (SRR)

- Replicates objects within the same region.
- Use for different accounts or prefixes.
- Helps with isolation and compliance.

Cross-Region Replication (CRR)

- Replicates objects to another region.
- Use for disaster recovery and latency.
- Supports replication of versions, delete markers and encryption.

IMPORTANT CONSIDERATIONS

- ✓ Use least privilege access.
- ✓ Enable encryption for all objects.
- ✓ Enable versioning for critical data.
- ✓ Design lifecycle policies to reduce cost.
- ✓ Understand request patterns and prefixes.
- ✓ Monitor with CloudWatch and S3 metrics.
- ✓ Test recovery regularly.

COMMON PITFALLS

- ✗ Publicly accessible buckets.
- ✗ Missing encryption.
- ✗ No versioning for important data.
- ✗ Wrong lifecycle policies.
- ✗ High request cost from bad patterns.
- ✗ Missing replication for DR.
- ✗ Not testing restore and recovery.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

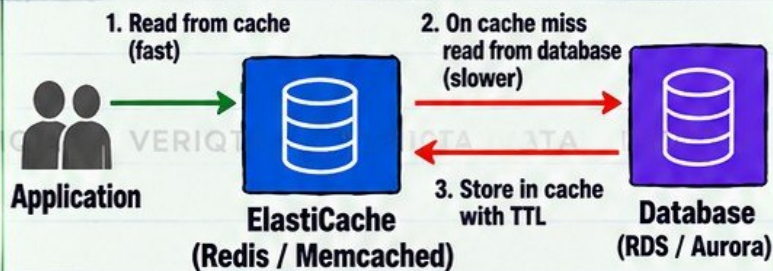


@veriqta

ElastiCache Failure Can Become Database Failure

Cache loss, stampedes, eviction, TTL design, hot keys, failover, and protecting the system of record.

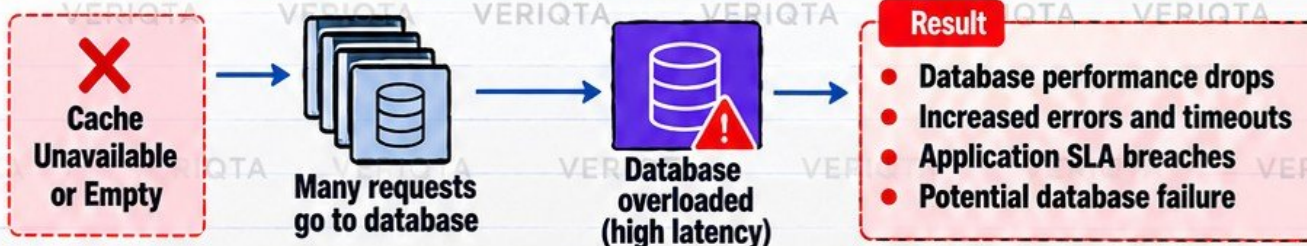
HOW ELASTICACHE WORKS



COMMON FAILURE SCENARIOS

- 1 Cache cluster failure (node loss, AZ issue).
- 2 Mass cache eviction (memory pressure).
- 3 Cache loss during maintenance or failover.
- 4 Application cache stampede after cache is empty.
- 5 Hot key overloads a single node.
- 6 Network or security group misconfiguration.
- 7 TTL misconfiguration (very low or no TTL).
- 8 ElastiCache service disruption or control plane issue.

CACHE STAMPEDE EFFECT



ELASTICACHE FAILOVER (REDIS EXAMPLE)



- ✓ Multi-AZ provides high availability.
- ✓ Failover promotes a replica to primary.
- ✓ Replication is asynchronous (possible data loss).
- ✓ Your application must handle brief connection errors and retry with backoff.

KEY DESIGN CONSIDERATIONS

- ✓ Set appropriate TTLs based on data freshness.
- ✓ Use jitter in TTL to avoid synchronized expiry.
- ✓ Implement cache warming for critical data.
- ✓ Use read-through or write-through patterns where suitable.
- ✓ Identify and mitigate hot keys (sharding, data modeling).
- ✓ Monitor memory usage, eviction rates and key space.
- ✓ Use Multi-AZ for production and configure automatic failover.
- ✓ Implement retries and fallback logic in the application.
- ✓ Protect the database with connection limits and queuing.

USEFUL AWS CLI COMMANDS

```

aws elasticache describe-cache-clusters # List clusters
aws elasticache describe-cache-nodes   # View node status
aws elasticache describe-replication-groups # Check replication group
aws cloudwatch get-metric-statistics    # View cache metrics
  
```

PRODUCTION LESSON

- ⚠ A cache outage can quickly overload your database.
- Design for cache failure, not just cache success.
- Use backoff, jitter and caching strategies to prevent stampedes.
- Monitor eviction rates and memory pressure.
- Keep the system of record protected with limits and scaling.

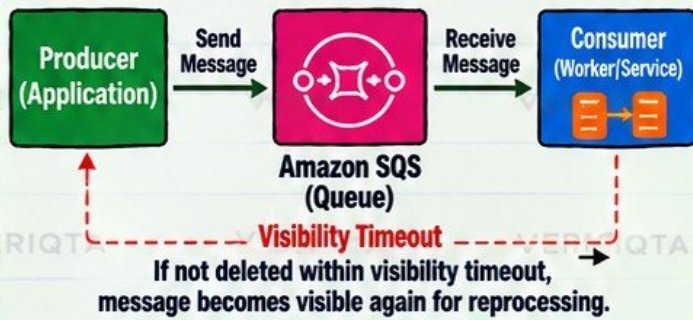
IMPORTANT METRICS TO MONITOR

- CPUUtilization # High CPU can lead to latency
- CurrConnections # Connection spikes
- CacheHits / CacheMisses # Miss spikes indicate issues
- Evictions # Keys being removed
- EngineCPUUtilization # Redis engine CPU
- ReplicationLag # Replica behind primary
- NetworkBandwidthIn/Out # Unexpected traffic spikes
- Database connections # Protect system of record

Queue Backlog as a Production Signal

SQS depth, message age, visibility timeout, poison messages, DLQs, consumer capacity, and backpressure.

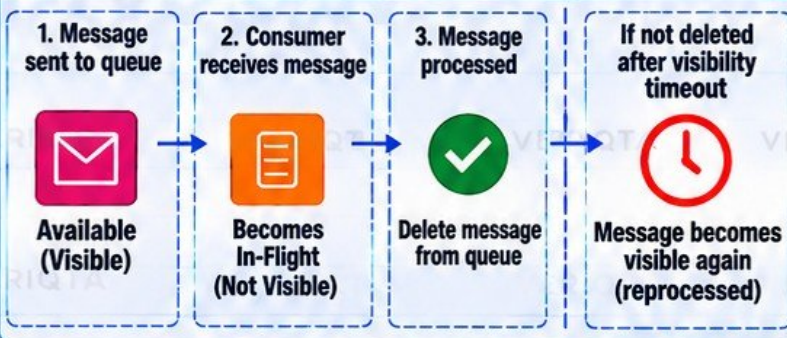
HOW SQS WORKS



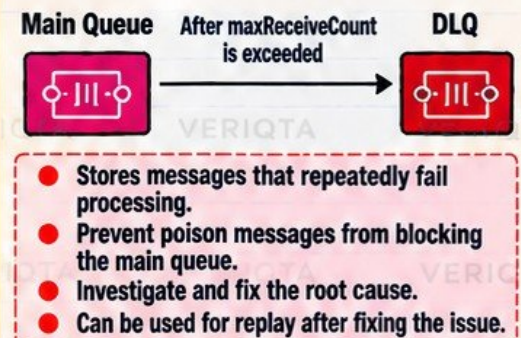
KEY SQS METRICS

Metric	What It Tells You
ApproximateNumberOfMessagesVisible	Current queue depth (backlog).
ApproximateNumberOfMessagesNotVisible	Messages being processed.
ApproximateAgeOfOldestMessage	How long messages have been waiting.
NumberOfMessagesSent	Incoming message rate.
NumberOfMessagesDeleted	Successful processing rate.
NumberOfMessagesReceived	Consumer receive rate.
NumberOfMessagesMovedToDeadLetterQueue	Poison or failed messages.

MESSAGE LIFECYCLE



DEAD LETTER QUEUE (DLQ)



COMMON CAUSES OF QUEUE BACKLOG

- 1 Consumer capacity is too low.
- 2 Downstream service is slow or unavailable.
- 3 Messages are large or complex to process.
- 4 Retries and redrives increasing queue depth.
- 5 Poison messages causing repeated failures.
- 6 Visibility timeout is too short or too long.
- 7 Sudden traffic spike (e.g. promotions, batch jobs).
- 8 Consumer errors, misconfiguration or crashes.

TROUBLESHOOTING STEPS

- 1 Check queue depth and message age.
- 2 Check consumer logs and error rate.
- 3 Validate downstream service health and latency.
- 4 Look for increased retries or DLQ messages.
- 5 Check if consumers are running and scaling.
- 6 Review visibility timeout and processing time.
- 7 Identify poison messages (check DLQ).
- 8 Increase consumer capacity or fix slow dependency.

DESIGN AND OPERATIONAL CONSIDERATIONS

- ✓ Set visibility timeout longer than your processing time.
- ✓ Use DLQ with an appropriate maxReceiveCount.
- ✓ Implement idempotent consumers.
- ✓ Monitor queue depth, message age and DLQ rate.
- ✓ Use autoscaling for consumers based on queue depth.
- ✓ Apply backpressure to upstream services when needed.
- ✓ Set alarms for message age and DLQ spikes.

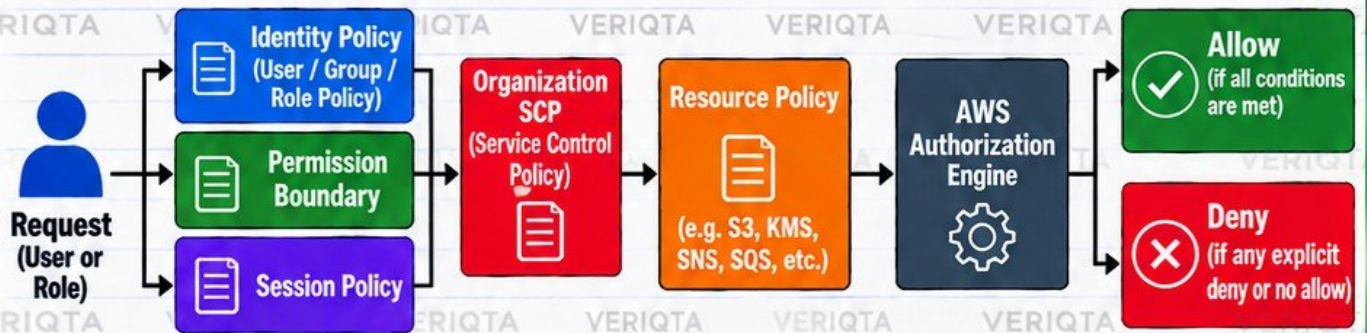
PRODUCTION INSIGHTS

- 💡 A growing message age is an early sign of downstream issues.
- 💡 Queue backlog can silently impact real-time user features.
- 💡 Retries can make the backlog worse if the root cause is not fixed.
- 💡 Monitor DLQ. A small DLQ rate is normal, but spikes indicate problems.
- 💡 Always design for backpressure and graceful degradation.

How AWS Authorization Actually Reaches a Decision

Identity policies, resource policies, SCPs, permission boundaries, sessions, explicit denies, and effective permissions.

AUTHORIZATION EVALUATION FLOW



POLICY TYPES AND PURPOSE

Policy Type	Purpose / What It Controls
Identity Policy	Permissions for users, groups or roles. Attached to the identity.
Permission Boundary	Sets the maximum permissions an identity can have.
Session Policy	Limits permissions for temporary credentials (e.g. via AssumeRole).
SCP	Sets the maximum permissions for accounts in an AWS Organization.
Resource Policy	Attached to a resource (e.g. S3, KMS) and defines who can access it.

AUTHORIZATION DECISION LOGIC

- 1 Check for any explicit deny (from any policy). If a deny exists, access is denied.
- 2 Evaluate the applicable SCP (if in an Organization). Permissions cannot exceed the SCP.
- 3 Evaluate the permission boundary (if set). Permissions cannot exceed the boundary.
- 4 Evaluate the identity policy. Must have an allow for the action.
- 5 Evaluate the session policy (if using temporary credentials). Further limits the permissions.
- 6 Evaluate the resource policy (if the resource supports it). Must also allow the action when required.
- 7 If all necessary policies allow and no explicit deny exists, the request is allowed.

EXAMPLE: S3 ACCESS DECISION



COMMON REASONS FOR ACCESS DENIED

- 1 Explicit deny in identity, resource policy or SCP.
- 2 SCP blocks the action at the account level.
- 3 Permission boundary is too restrictive.
- 4 Session policy removes required permissions.
- 5 Missing allow in identity or resource policy.
- 6 Accessing resource in another account without proper resource policy.
- 7 Conditions not met (e.g. IP, MFA, VPC endpoint).

DEBUGGING ACCESS DENIED

- 1 Check the error message and AWS request ID.
- 2 Review the IAM policy simulator.
- 3 Check identity policies, permission boundary and session policy.
- 4 Verify SCPs if using AWS Organizations.
- 5 Check the resource policy (e.g. S3 bucket policy)
- 6 Look at CloudTrail for detailed authorization decision.
- 7 Validate any conditions (MFA, IP, VPC endpoint, tags, etc.).

KEY TAKEAWAYS

- ✓ Explicit deny always wins.
- ✓ Effective permissions are the intersection of all applicable policies.
- ✓ SCPs and permission boundaries set the maximum permissions.
- ✓ Resource policies are required for many cross-account scenarios.
- ✓ Temporary credentials are further limited by session policies.
- ✓ Use the IAM policy simulator and CloudTrail for troubleshooting.
- ✓ Always design with least privilege.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

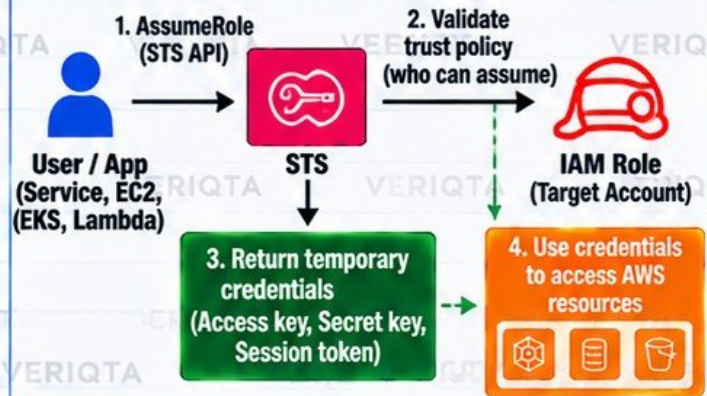
IAM Role Assumption in Production

STS, temporary credentials, trust policies, role chaining, workload identity, and cross-account access.

WHAT IS ROLE ASSUMPTION

- AWS Security Token Service (STS) issues temporary credentials for an IAM role.
- You cannot assume a role unless the trust policy allows it.
- Temporary credentials include an access key, secret key and session token.
- Role assumption is used for cross-account access, workload identity and temporary privilege elevation.
- The permissions you get come from the role's identity policies and permission boundaries (if any).

HOW ROLE ASSUMPTION WORKS



TRUST POLICY (WHO CAN ASSUME)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::111122223333:role/AppRole" },
      "Action": "sts:AssumeRole"
    }
  ]
}
  
```

- Defines which principal (user, role, service or account) can assume the role.
- Can include conditions (MFA, SourceArn, ExternalId, etc).
- Use least privilege. Avoid using "*".

ASSUME ROLE CLI EXAMPLE

```

aws sts assume-role \
--role-arn arn:aws:iam::222233334444:role/ProdRole \
--role-session-name prod-session
  
```

RESPONSE (TEMPORARY CREDENTIALS)

```

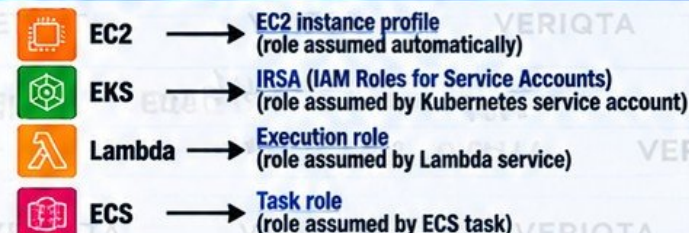
{
  "Credentials": {
    "AccessKeyId": "ASIA...",
    "SecretAccessKey": "wJalrXUtnFEMI/K7MDENG...",
    "SessionToken": "IQoJb3JpZ2luX2VjE...",
    "Expiration": "2026-10-01T12:00:00Z"
  }
}
  
```

ROLE CHAINING

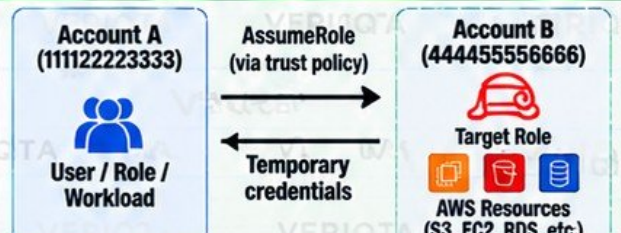


- You can chain roles (Role A → B → C).
- Each role must trust the previous role.
- The maximum session duration for chained roles is 1 hour.
- Use session names for traceability.

WORKLOAD IDENTITY EXAMPLES



CROSS-ACCOUNT ACCESS



KEY TAKEAWAYS

- ✓ STS provides temporary credentials, not long-term access.
- ✓ Trust policy controls who can assume the role.
- ✓ Use least privilege and conditions (MFA, ExternalId, SourceArn).
- ✓ Role chaining max session duration is 1 hour.
- ✓ Use workload identity instead of hardcoded credentials.
- ✓ Always log and monitor role assumption in CloudTrail.

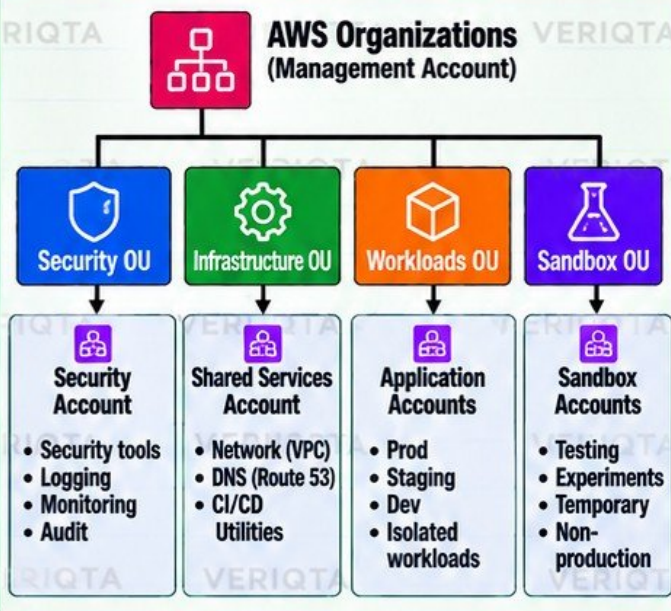
COMMON ISSUES AND FIXES

- 1 Access denied → Check trust policy and principal ARN.
- 2 Expired token → Refresh credentials before expiration.
- 3 Cannot assume role → Verify permissions and conditions.
- 4 Role chaining limit → Max 1 hour for chained roles.
- 5 Wrong account → Check role ARN and target account ID.
- 6 Use CloudTrail → Investigate AssumeRole events.

AWS Multi-Account Security Architecture

Organizations, account boundaries, SCPs, centralized security, workload separation, and blast-radius reduction.

MULTI-ACCOUNT ARCHITECTURE OVERVIEW



SERVICE CONTROL POLICIES (SCPs)



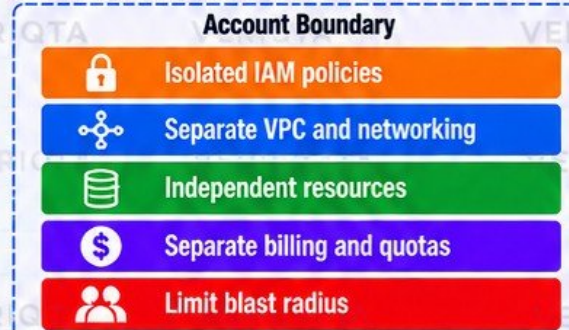
- ✓ Define what services can be used.
- ✓ Explicitly deny risky actions.
- ✓ Prevent leaving the approved region.
- ✓ Block unapproved instance types.
- ✓ Enforce security and compliance rules.
- ✓ Apply to OUs or individual accounts.

EXAMPLE SCP (DENY UNAPPROVED REGIONS)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": ["us-east-1", "us-west-2"]
        }
      }
    }
  ]
}
```

This SCP denies all actions in regions outside us-east-1 and us-west-2.

ACCOUNT BOUNDARIES AND ISOLATION



CENTRALIZED SECURITY SERVICES



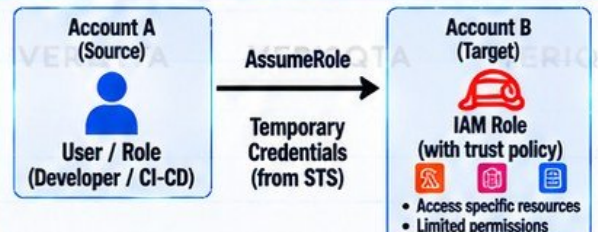
WORKLOAD SEPARATION STRATEGY

- ✓ Separate production, staging and development accounts.
- ✓ Use different accounts for different business units or teams.
- ✓ Isolate high-risk or experimental workloads in sandbox accounts.
- ✓ Use distinct accounts for regulated or sensitive data.
- ✓ Control access between accounts using IAM roles and SCPs.

KEY TAKEAWAYS

- ✓ Use AWS Organizations to manage multiple accounts.
- ✓ Apply SCPs to set security guardrails.
- ✓ Isolate workloads using separate accounts.
- ✓ Centralize security, logging and monitoring.
- ✓ Use cross-account roles for controlled access.
- ✓ Multi-account architecture reduces blast radius and improves security.

CROSS-ACCOUNT ACCESS



COMMON MISTAKES TO AVOID

- 1 Keeping all workloads in a single account.
- 2 Not using SCPs or having weak guardrails.
- 3 Giving excessive cross-account permissions.
- 4 Mixing production and development workloads.
- 5 Not centralizing logging and security monitoring.
- 6 Failing to isolate high-risk or experimental workloads.

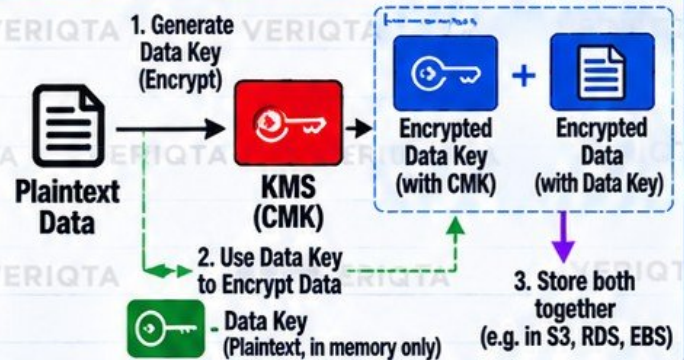
KMS and the Encryption Dependency Nobody Notices

Key policies, grants, IAM interaction, envelope encryption, service integrations, throttling, and key availability.

WHAT IS AWS KMS?

- ✓ AWS Key Management Service (KMS) helps you create, manage and use cryptographic keys.
- ✓ Used for encryption, decryption, signing and verifying data.
- ✓ Integrated with many AWS services (S3, EBS, RDS, Lambda, EKS, etc.).
- ✓ Access is controlled using key policies, IAM policies and grants.
- ✓ KMS is a critical dependency. If KMS is unavailable, applications that use it can fail.

HOW ENVELOPE ENCRYPTION WORKS



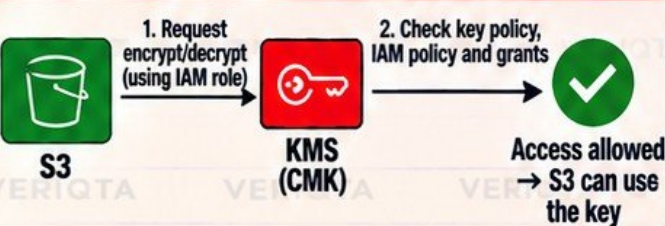
KMS KEY TYPES

Customer Managed Key (CMK)	AWS Managed Key (AWS Owned)	AWS Owned Key
 • Full control • Custom policies • Manual or automatic rotation • Used for most production workloads	 • Managed by AWS • Used by AWS services • No policy changes • Free to use	 • Used for AWS internal operations • Not visible in your account • No customer control

KEY POLICY vs IAM POLICY vs GRANTS

Key Policy	IAM Policy	Grant
 • Resource-based policy attached to the key. • Defines who can use the key. • Must allow access before IAM policies can grant permissions. • Often the most important policy.	 • Identity-based policy attached to a user, role or group. • Grants permissions (e.g. kms:Encrypt, kms:Decrypt). • Works together with the key policy.	 • Temporary permission to use the key. • Often created automatically by AWS services. • Used by services like RDS, S3, EBS, Lambda. • Can be time-limited and revoked.

EXAMPLE: S3 ENCRYPTION WITH KMS



COMMON KMS DEPENDENCIES

 S3	 EBS	 RDS	 Lambda	 EKS
 ECS	 CloudWatch (Logs)	 CloudTrail	 Secrets Manager	 Systems Manager

KMS THROTTLING AND LIMITS

- ⚠ KMS has request rate limits (TPS) per account per region.
- Sudden traffic spikes can cause ThrottlingException.
- Common with high-volume encryption (e.g. many Lambda functions starting at once).
- Use data key caching, exponential backoff and retries.
- Monitor KMS metrics in CloudWatch (Throttles, Latency, Errors).

KEY AVAILABILITY AND FAILURE IMPACT

- ✗ If KMS is unavailable, services that rely on it can fail.
- Examples: cannot launch new EC2 instances, cannot encrypt to S3, cannot access encrypted RDS.
- Use multi-AZ AWS KMS (already designed for high availability).
- Implement retries and graceful degradation in applications.
- Have a recovery plan and monitor KMS health.

BEST PRACTICES

- ✓ Use least privilege in key policies and IAM policies.
- ✓ Separate keys by environment (dev, staging, prod).
- ✓ Enable key rotation (automatic rotation where possible).
- ✓ Use grants for AWS services instead of broad permissions.
- ✓ Monitor KMS usage, throttling and errors.
- ✓ Understand all services that depend on each key.

COMMON MISTAKES TO AVOID

- 1 Key policy does not allow the IAM principal.
- 2 Overly permissive key policies (wildcards).
- 3 Forgetting service integrations that need access (e.g. RDS, S3).
- 4 Not handling throttling and retries.
- 5 Deleting or disabling a key that is still in use.
- 6 No monitoring or alerting for KMS errors and throttling.

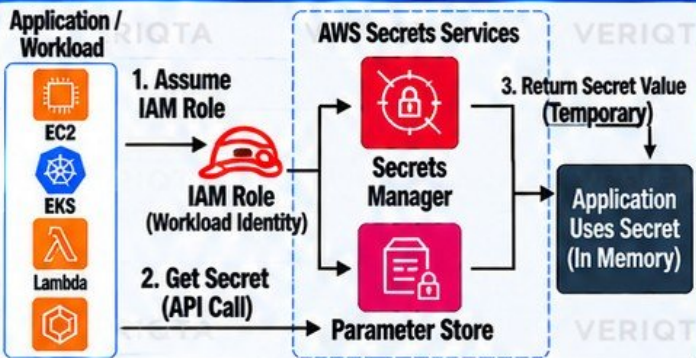
Secrets Without Permanent Credentials

Secrets Manager, Parameter Store, workload identity, rotation, caching, access boundaries, and auditability.

WHY USE AWS SECRETS MANAGEMENT?

- ✓ Avoid hardcoding credentials in code, images or configs.
- ✓ Use temporary access instead of long-lived credentials.
- ✓ Centralized and secure storage of sensitive data.
- ✓ Supports automatic rotation and auditing.
- ✓ Integrates with IAM, services and workload identity.
- ✓ Reduces risk of credential leakage and unauthorized access.

HIGH-LEVEL ARCHITECTURE



SECRETS MANAGER vs PARAMETER STORE

AWS Secrets Manager



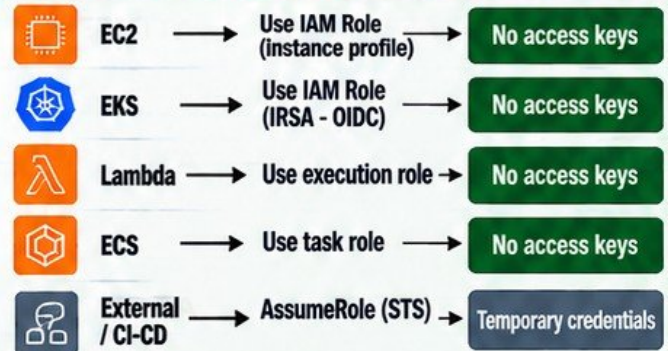
- Designed for sensitive secrets (databases, API keys, etc.)
- Automatic rotation support
- Built-in secret versioning
- Fine-grained access control
- Higher cost (but more features)

Parameter Store



- Used for configuration data (including sensitive data)
- No automatic rotation (manual rotation only)
- Cheaper
- Supports parameter tiers (Standard, Advanced)
- Good for non-database secrets

WORKLOAD IDENTITY (NO PERMANENT CREDENTIALS)



SECRET ROTATION

Automatic Rotation



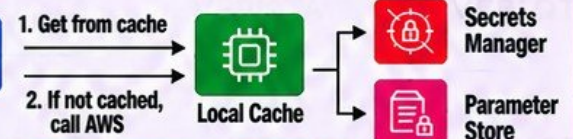
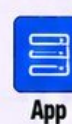
- Rotate database credentials automatically.
- Use AWS managed rotation or custom Lambda.
- Minimizes impact with seamless rotation.
- Applications should handle credential changes (use connection pooling and retries).



Manual Rotation

- Used when automatic rotation is not supported.
- Follow a defined rotation process.
- Update dependent systems after rotation.

CACHING SECRETS



- Cache secrets in memory to reduce API calls and latency.
- Use appropriate TTL (time-to-live).
- Handle cache refresh and failures gracefully.
- Do not store secrets in logs or persistent disk.

ACCESS BOUNDARIES AND SECURITY

- | Policy Type | Key Features |
|-----------------------------|---|
| IAM Policies | <ul style="list-style-type: none"> Define who can access specific secrets. Use least privilege permissions. Restrict actions (GetSecretValue, DescribeSecret, etc.). |
| Resource Policies | <ul style="list-style-type: none"> Control cross-account access. Use conditions (e.g. source VPC, source account). |
| SCPs (Organizations) | <ul style="list-style-type: none"> Prevent secrets access outside allowed accounts. Enforce security guardrails. Block risky actions at the organization level. |

AUDITABILITY AND MONITORING

- | Service | Key Features |
|-------------------|--|
| CloudTrail | <ul style="list-style-type: none"> Logs all API calls (GetSecretValue, PutSecretValue, etc.). Track who accessed secrets, when and from where. |
| CloudWatch | <ul style="list-style-type: none"> Monitor API errors and throttling. Create alarms for unusual access patterns. |
| AWS Config | <ul style="list-style-type: none"> Track changes to secret resource policies. Monitor compliance and configuration drift. |

KEY TAKEAWAYS

- ✓ Use Secrets Manager or Parameter Store, not hardcoded credentials. Prefer workload identity and IAM roles over long-lived access keys.
- ✓ Enable automatic rotation for supported services.
- ✓ Cache secrets to reduce latency and API calls.
- ✓ Use least privilege, resource policies and SCPs.
- ✓ Monitor access with CloudTrail and CloudWatch.

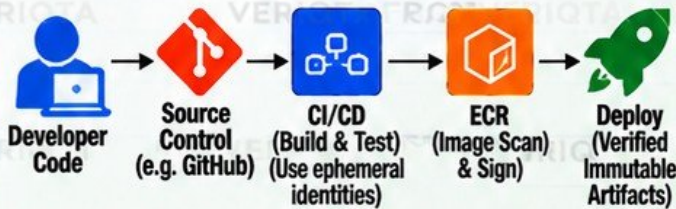
COMMON MISTAKES TO AVOID

1. Hardcoding secrets in code, images or environment variables.
2. Using long-lived IAM users or access keys.
3. Not enabling rotation for database credentials.
4. Giving overly broad permissions (e.g. secretsmanager:*).
5. Not monitoring secret access and API errors.
6. Storing secrets in logs, code repositories or local files.

AWS Supply-Chain Security

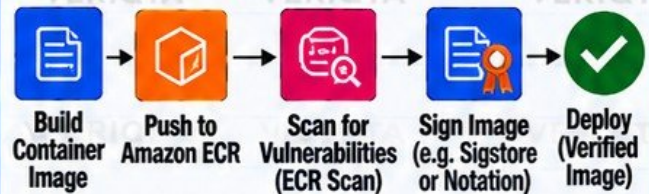
Artifact provenance, ECR, image scanning, CI/CD identities, immutable artifacts, signing, dependency risk, and deployment trust.

SUPPLY-CHAIN SECURITY OVERVIEW



- ✓ Ensure code, dependencies and container images are trusted.
- ✓ Use short-lived credentials for CI/CD.
- ✓ Scan and sign artifacts before deployment.
- ✓ Keep artifacts immutable and traceable.
- ✓ Detect and prevent tampering across the pipeline.

CONTAINER IMAGE SUPPLY CHAIN (ECR)



- Use Amazon ECR for private container registries.
- Enable image scanning (basic or enhanced).
- Sign images using Sigstore, Notation or similar tools.
- Enforce policies to allow only signed and scanned images.
- Use immutable tags (avoid latest).

CI/CD IDENTITIES (NO LONG-TERM CREDENTIALS)



- Use OIDC to authenticate your CI/CD pipeline.
- Avoid long-lived IAM users or access keys.
- Grant least privilege permissions.
- Use separate roles for build, test and deploy.
- Apply conditions (repo, branch, environment, etc.).

ARTIFACT SIGNING AND VERIFICATION



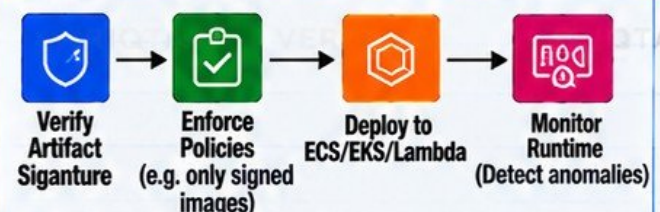
- Sign container images and other artifacts (binaries, packages).
- Store signatures with the artifact.
- Verify signatures during deployment.
- Reject unsigned or tampered artifacts.
- Use trusted certificate authorities or keyless signing (Sigstore).

DEPENDENCY RISK MANAGEMENT



- Scan dependencies for known vulnerabilities (SCA tools).
- Use private package repositories when required.
- Enforce approved versions and license policies.
- Monitor for new CVEs and update dependencies.
- Remove unused or risky dependencies.

DEPLOYMENT TRUST AND RUNTIME PROTECTION



- Verify signatures before deployment.
- Use admission controllers (e.g. Kyverno, OPA) in Kubernetes.
- Enforce policies to block unsigned or untrusted images.
- Monitor runtime for unusual behaviour.
- Have rollback and incident response procedures.

KEY TAKEAWAYS

- ✓ Use ephemeral CI/CD identities (OIDC, AssumeRole).
- ✓ Scan, sign and verify all artifacts.
- ✓ Keep artifacts immutable and traceable.
- ✓ Manage and scan dependencies.
- ✓ Enforce strong deployment policies.
- ✓ Continuously monitor for supply-chain threats.

COMMON MISTAKES TO AVOID

- 1 Using long-lived AWS access keys in CI/CD.
- 2 Not scanning container images or dependencies.
- 3 Allowing unsigned or untrusted artifacts in production.
- 4 Using mutable tags like latest.
- 5 Giving excessive permissions to CI/CD roles.
- 6 Not monitoring for new vulnerabilities or tampering.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

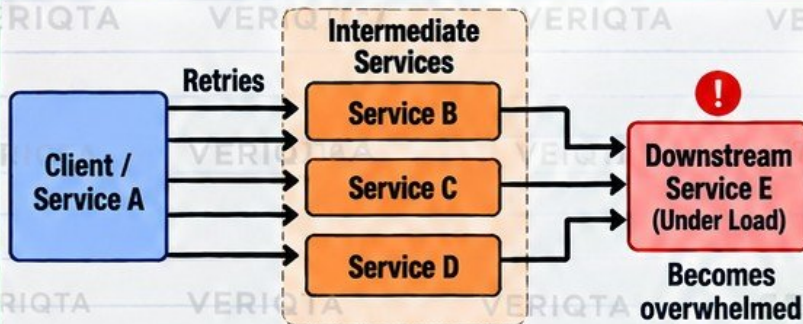


@veriqta

Retries Can Take Production Down

Retry amplification, exponential backoff, jitter, timeouts, dependency saturation, and retry budgets.

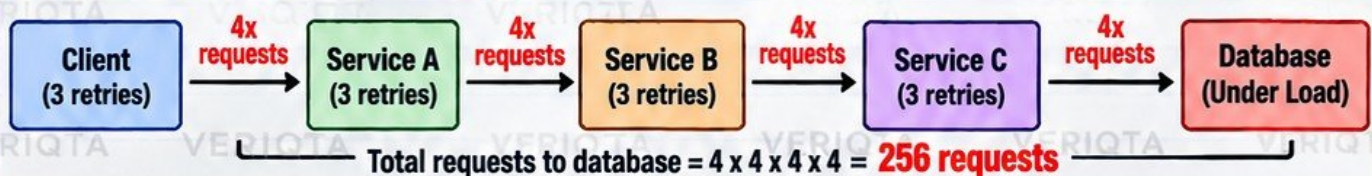
HOW RETRY AMPLIFICATION HAPPENS



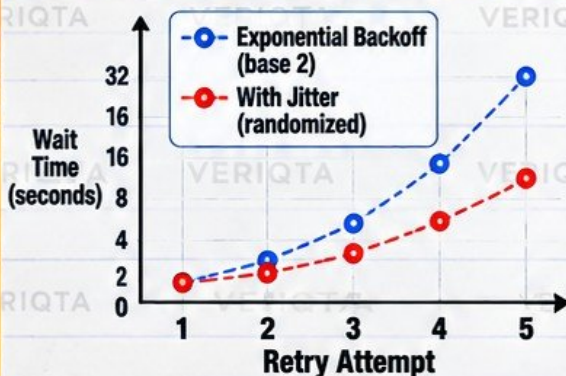
THE PROBLEM

- 1 Multiple layers retry the same request.
- 2 Each layer increases the total number of requests (retry amplification).
- 3 Downstream service becomes saturated.
- 4 Latency increases, causing more retries.
- 5 A small issue can turn into a full outage.

EXAMPLE: RETRY AMPLIFICATION



EXPONENTIAL BACKOFF WITH JITTER



Why Jitter?

- Prevents all clients retrying at the same time.
- Reduces traffic spikes.
- Helps avoid overwhelming a recovering service.

BEST PRACTICES

- 1 Use exponential backoff with jitter.
- 2 Set reasonable retry limits.
- 3 Use appropriate timeouts.
- 4 Avoid retrying non-retryable errors (e.g. 4xx).
- 5 Implement circuit breakers.
- 6 Use retry budgets.
- 7 Monitor retry rates and downstream saturation.
- 8 Design idempotent operations for safe retries.

CODE EXAMPLE (AWS SDK – PYTHON)

```

from botocore.config import Config
import boto3

config = Config(
    retries={
        'max_attempts': 5,
        'mode': 'adaptive' # uses exponential backoff + jitter
    },
    connect_timeout=3,
    read_timeout=10
)

s3 = boto3.client('s3', config=config)
  
```

Notes:

- Adaptive mode uses exponential backoff with jitter.
- Set timeouts to avoid hanging requests.
- Adjust max_attempts based on service and use case.

KEY TAKEAWAYS

- ✓ Retries are necessary but can cause outages if misconfigured.
- ✓ Use exponential backoff with jitter.
- ✓ Set clear retry limits and timeouts.
- ✓ Understand the impact across all layers of your architecture.
- ✓ Monitor retry rates, latency, and downstream saturation.
- ✓ Use circuit breakers and retry budgets to protect critical services.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Designing for Partial Failure

Graceful degradation, dependency isolation, circuit breaking, queues, fallbacks, and preserving critical paths.

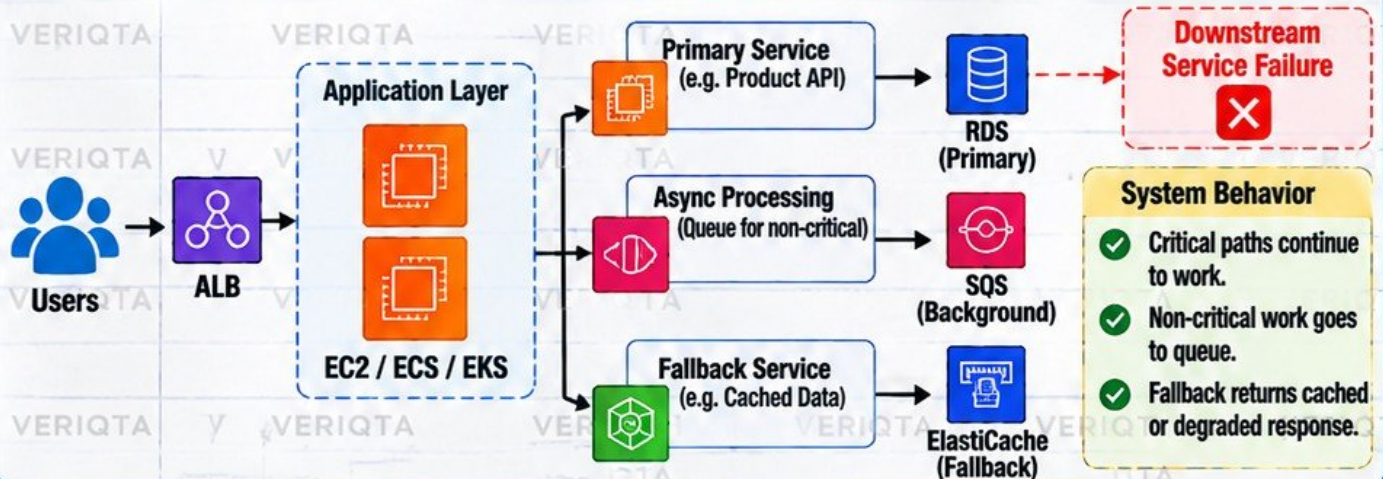
WHAT IS PARTIAL FAILURE?

- A situation where one or more components fail, but the entire system does not go down.
- Common in distributed systems and cloud environments.
- The goal is to keep critical functionality working while isolating and mitigating the impact of failures.

WHY IT HAPPENS IN AWS

- 1 A downstream service becomes unavailable.
- 2 Network or DNS issues between services.
- 3 Throttling or rate limits.
- 4 Regional or Availability Zone issues.
- 5 A dependency experiences high latency or errors.

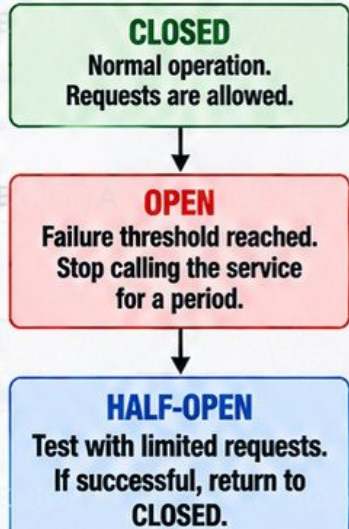
EXAMPLE ARCHITECTURE FOR PARTIAL FAILURE RESILIENCE



RESILIENCE PATTERNS

- 1 **Graceful Degradation**
Return a limited or simplified response when a dependency fails.
- 2 **Dependency Isolation**
Use separate services, accounts, or queues to limit blast radius.
- 3 **Circuit Breaking**
Stop calling a failing service after a threshold of failures.
- 4 **Queues and Async Processing**
Move non-critical work to queues to avoid blocking the main flow.
- 5 **Fallbacks**
Use cached data, default responses, or alternative providers.
- 6 **Timeouts and Retries**
Use sane timeouts, limited retries, and backoff.

CIRCUIT BREAKER STATES



KEY TAKEAWAYS

- 1 Design for partial failure from day one.
- 2 Protect critical user paths.
- 3 Use circuit breakers, timeouts and fallbacks.
- 4 Isolate dependencies and use queues for non-critical work.
- 5 Monitor errors, latency and saturation on all downstream services.
- 6 Test failure scenarios regularly (e.g. game days).

REAL WORLD EXAMPLE

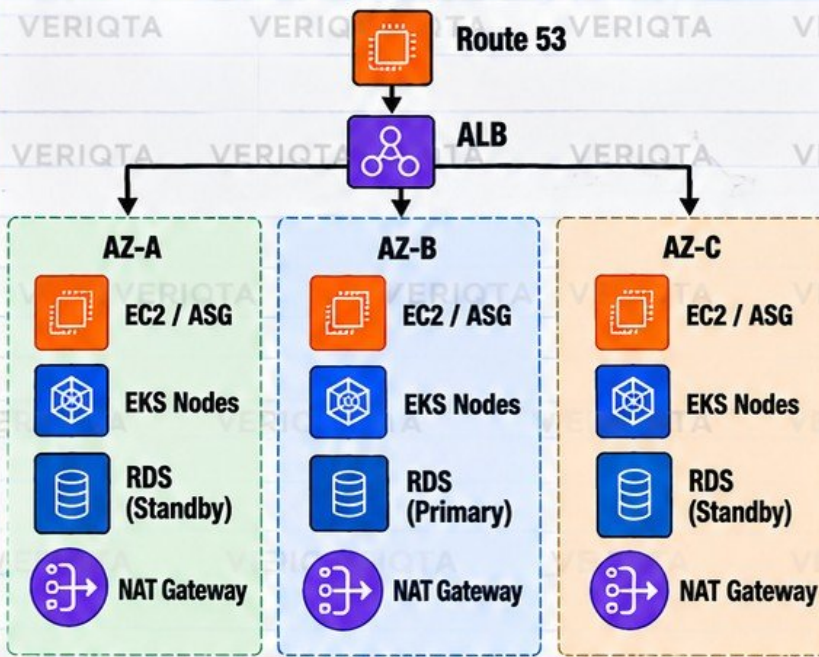
If a payment service is down:

- Allow users to browse products.
- Disable checkout or use an alternative provider.
- Show a clear message instead of failing the entire application.

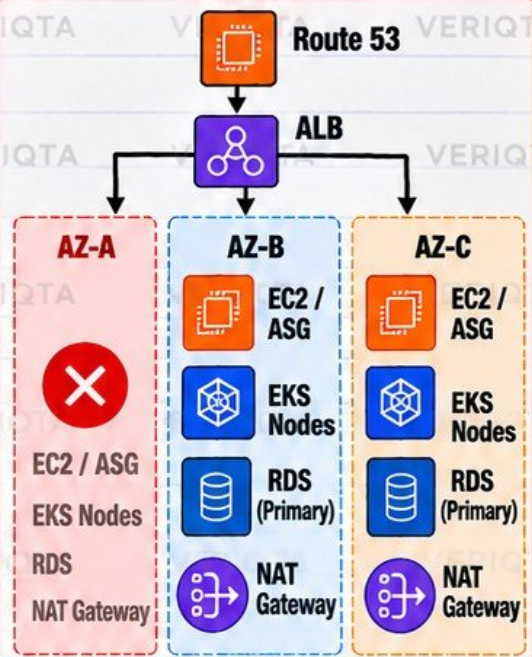
What Happens When an Availability Zone Fails?

Capacity redistribution, load balancing, databases, NAT, Kubernetes, Auto Scaling, and hidden single-AZ dependencies.

MULTI-AZ ARCHITECTURE (BEFORE AZ FAILURE)



AFTER AZ-A FAILURE



WHAT HAPPENS?

- Resources in AZ-A become unavailable.
- ALB stops sending traffic to unhealthy targets.
- Auto Scaling launches new instances in healthy AZs.
- RDS Multi-AZ fails over to a standby in another AZ.
- EKS reschedules pods to healthy nodes.
- NAT in the failed AZ is lost, traffic uses NAT in remaining AZs.
- Traffic continues if there is enough capacity.

POTENTIAL IMPACT

- Increased latency**
More traffic to remaining AZs.
- Service degradation**
If capacity is insufficient.
- Database failover**
Short downtime during RDS failover.
- Pod rescheduling delays**
If cluster has limited nodes.
- NAT gateway loss**
Can affect outbound traffic.
- Cost increase**
Due to rebalancing and higher load.
- Hidden single-AZ dependencies**
Can still cause full or partial outage.

HIDDEN SINGLE-AZ DEPENDENCIES

- NAT gateway only in one AZ.
- RDS not configured for Multi-AZ.
- EFS mounted in one AZ.
- EKS with insufficient nodes in other AZs.
- Stateful workloads not replicated.
- Hardcoded AZ-specific endpoints.
- Third-party services with single-AZ deployment.
- Data stored only in one AZ (e.g. cache, queue, logs).

KEY DESIGN CONSIDERATIONS

- Use Multi-AZ for critical services (RDS, EFS, ElastiCache).
- Distribute workloads across multiple AZs.
- Ensure sufficient Auto Scaling capacity.
- Use multiple NAT gateways (one per AZ).
- Test AZ failure scenarios regularly.
- Identify and remove single-AZ dependencies.

REAL WORLD LESSONS

- Multi-AZ does not mean no impact.
- Capacity must be pre-provisioned in other AZs.
- Failovers cause temporary latency and errors.
- Some services recover automatically, others do not.
- Hidden dependencies can still take down the system.
- Test AZ failures to validate your design and runbooks.

KEY TAKEAWAYS

- An AZ failure is expected, not rare.
- Your architecture must continue to work.
- Design for automatic recovery and capacity redistribution.
- Monitor cross-AZ capacity, latency, and errors.
- Eliminate single-AZ dependencies.
- Regularly test your resilience.



@veriqta



@veriqta



@veriqta



@veriqta



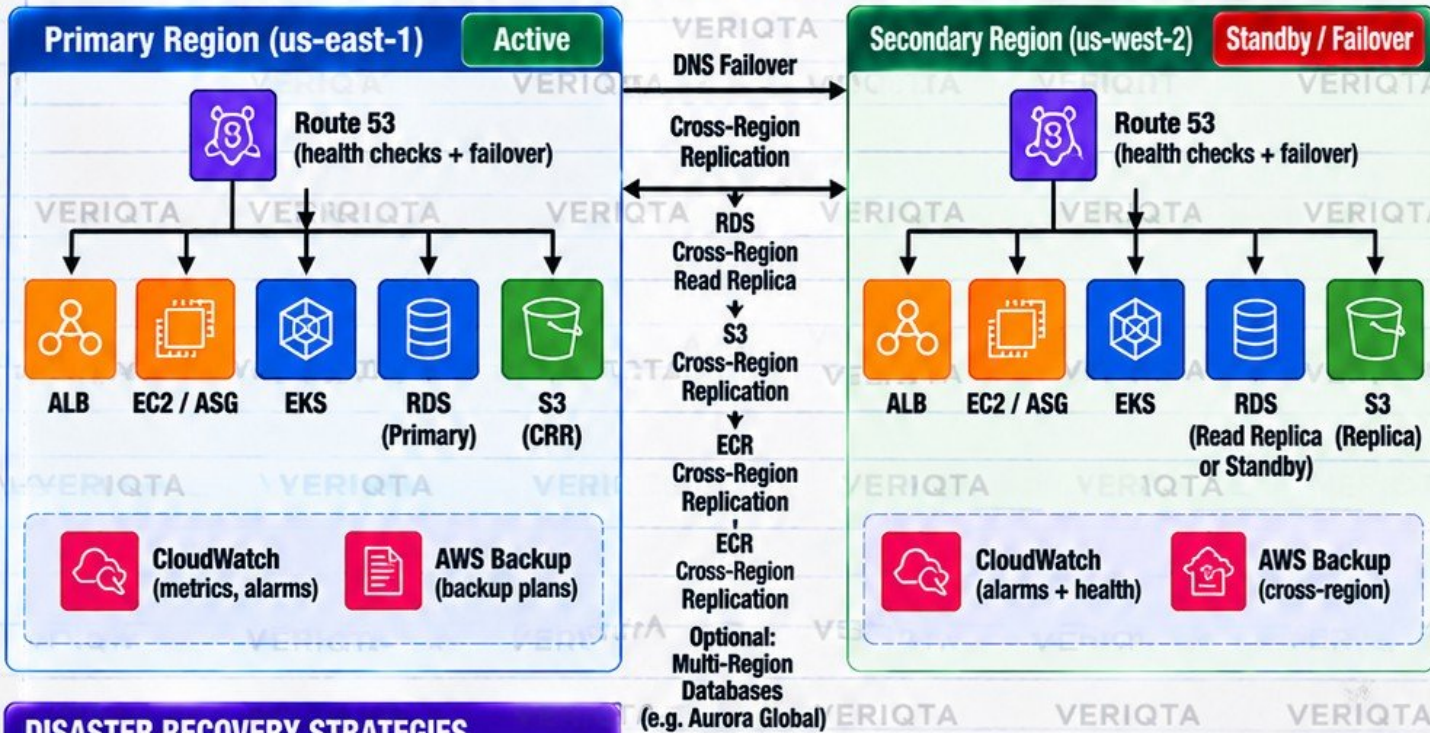
@veriqta



@veriqta

Regional Failure and Disaster Recovery

Backup and restore, pilot light, warm standby, active-active, RTO, RPO, data replication, DNS, and operational complexity.



DISASTER RECOVERY STRATEGIES

Strategy	Architecture	RTO	RPO	Use Case	Pros	Cons
Backup and Restore		Hours – Days	Hours – Days	Non-critical workloads	Lowest cost	Longest downtime
Pilot Light		Hours	Minutes – Hours	Small to medium workloads	Lower cost	Some setup required
Warm Standby		Minutes – Hours	Minutes	Business-critical workloads	Faster recovery	Higher cost than pilot light
Active-Active		Seconds – Minutes	Seconds – Minutes	Mission-critical workloads	Highest availability	Highest cost and complexity

KEY COMPONENTS IN A REGIONAL FAILURE

- 1 DNS failover (Route 53 health checks and routing).
- 2 Compute (EC2 Auto Scaling or EKS in standby/active).
- 3 Databases (RDS cross-region, Aurora Global Database).
- 4 Storage (S3 cross-region replication).
- 5 Backups (cross-region backup plans).
- 6 Networking (VPC, Transit Gateway, Direct Connect).
- 7 IAM and security (replicate configuration and access).
- 8 Monitoring (CloudWatch, alarms, synthetic checks).

RTO vs RPO

RTO
Recovery Time Objective
Maximum acceptable time to restore service after a failure.

RPO
Recovery Point Objective
Maximum acceptable data loss measured in time.

KEY TAKEAWAYS

- ✓ A regional failure is rare but real.
- ✓ Choose a DR strategy based on business impact, not cost only.
- ✓ Test your disaster recovery regularly.
- ✓ Replicate data, not just infrastructure.
- ✓ Automate failover where possible.
- ✓ Consider operational complexity and runbooks.
- ✓ Validate DNS, networking, IAM, and dependencies during tests.
- ✓ Monitor cross-region replication and recovery readiness.

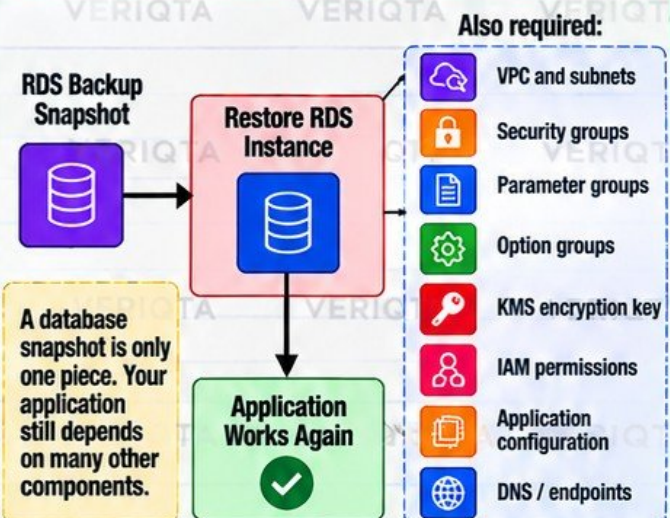
Backups Are Not Recovery

Restore testing, dependency reconstruction, encryption keys, configuration, data integrity, recovery sequencing, and evidence.

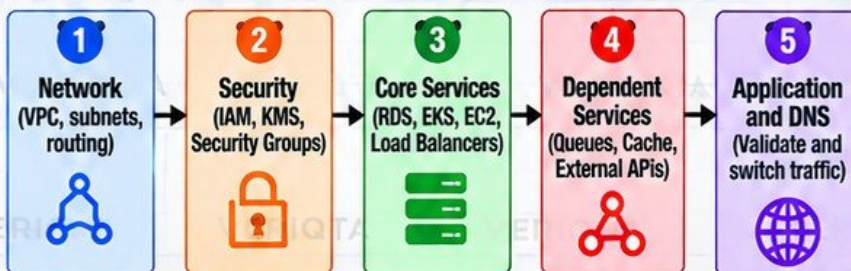
BACKUP VS RECOVERY

Backup	Recovery
✗ A copy of your data or configuration. ✗ Stored for future use. ✗ Does not guarantee your application will work when restored. ✗ Does not include dependent services.	✓ Restoring data and rebuilding everything needed to run. ✓ Includes infrastructure, configuration, dependencies and access. ✓ Must be tested regularly. ✓ Proves that your system can actually work again.

REAL EXAMPLE: RDS RESTORE IS NOT COMPLETE



RECOVERY SEQUENCING (EXAMPLE)



COMMON RECOVERY FAILURES

- 1 Backup exists but restore was never tested.
- 2 Missing encryption keys (KMS).
- 3 Missing configuration and parameter groups.
- 4 Dependent services not restored.
- 5 IAM roles and permissions not recreated.
- 6 Application fails after database restore.
- 7 DNS not updated or tested.
- 8 Data appears restored but is incomplete or corrupted.

DATA INTEGRITY CHECKS

- ✓ Verify record counts and checksums.
- ✓ Validate application data (not just database).
- ✓ Check for data corruption after restore.
- ✓ Compare backups with live data samples.
- ✓ Test application functionality with real data.
- ✓ Validate logs, indexes and metadata.

RESTORE TESTING CHECKLIST

- 1 Perform regular restore tests.
- 2 Test in an isolated environment.
- 3 Validate all dependencies.
- 4 Test application end-to-end.
- 5 Measure RTO and RPO.
- 6 Document the complete recovery process.
- 7 Keep evidence of successful tests.
- 8 Update runbooks after each test.

EVIDENCE OF RECOVERY

- ✓ Application is fully functional.
- ✓ All critical data is present and valid.
- ✓ Dependent services are working.
- ✓ Monitoring and alerting are healthy.
- ✓ RTO and RPO targets are met.
- ✓ Test results and logs are documented.
- ✓ Runbooks are updated and verified.

KEY TAKEAWAYS

- ✓ A backup is not a recovery plan.
- ✓ You must test restores regularly.
- ✓ Recovery includes infrastructure, configuration, dependencies and access.
- ✓ Encryption keys and IAM roles are critical.
- ✓ Validate data integrity, not just that the restore worked.
- ✓ Follow a clear recovery sequence.
- ✓ Keep evidence and update runbooks.

REAL WORLD SCENARIO

A company had RDS backups but never tested a full restore. During an incident, they restored the database but the application still failed because the parameter group, security groups and KMS key were not recreated. DNS was also still pointing to the old endpoint.

Result: several hours of downtime.

Lesson: backups alone are not recovery. You must test the complete recovery process.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CloudWatch Is Not an Observability Strategy

Metrics, logs, traces, application telemetry, high-cardinality dimensions, dashboards, alarms, and blind spots.

WHAT CLOUDWATCH PROVIDES



Metrics

Infrastructure and service metrics



Logs

Logs from AWS services, EC2, EKS, Lambda and applications



Traces

X-Ray for distributed tracing



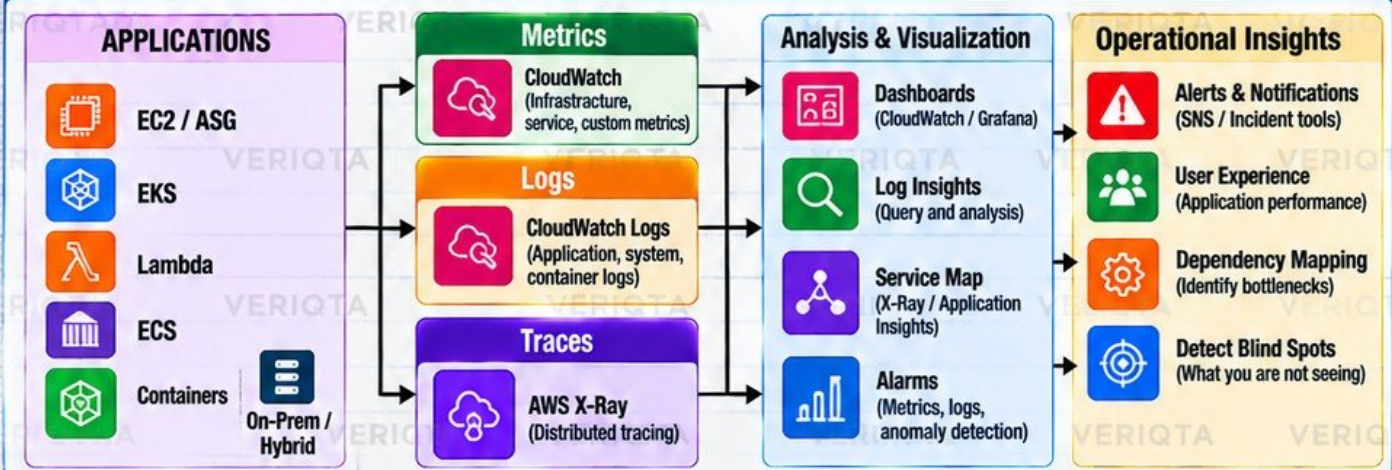
Alarms

Thresholds and notifications

WHY IT'S NOT ENOUGH

- 1 No built-in full application observability.
- 2 Limited support for high-cardinality metrics.
- 3 Traces require X-Ray and proper instrumentation.
- 4 Logs need structure, correlation and context.
- 5 No automatic service dependency mapping.
- 6 Dashboards can be misleading.
- 7 Important blind spots (application, user, network).

A COMPLETE OBSERVABILITY ARCHITECTURE ON AWS



HIGH-CARDINALITY DIMENSIONS

- ✓ Default CloudWatch metrics are low-cardinality.
- ✓ High-cardinality metrics (e.g. per user, per request, per pod) increase cost and require proper design.
- ✓ Use aggregation, sampling and meaningful dimensions.
- ✓ Avoid unnecessary high-cardinality labels.
- ✓ Monitor cardinality and cost.

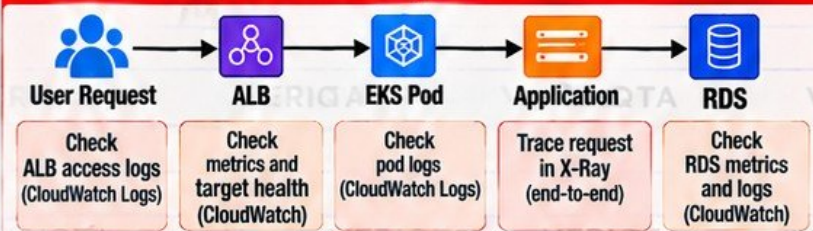
COMMON BLIND SPOTS

- 1 Application-level errors not in logs.
- 2 High latency without errors.
- 3 Downstream service failures.
- 4 Dependencies outside AWS.
- 5 User experience issues.
- 6 Missing traces or incomplete trace context.
- 7 No correlation between metrics, logs and traces.

BEST PRACTICES

- ✓ Use metrics, logs and traces together.
- ✓ Instrument your application.
- ✓ Use meaningful and consistent labels.
- ✓ Build clear dashboards for SLOs and SLIs.
- ✓ Set actionable alarms (not noisy).
- ✓ Use X-Ray or OpenTelemetry.
- ✓ Monitor dependencies and external services.
- ✓ Regularly review blind spots.

EXAMPLE: TRACE A FAILED REQUEST



KEY TAKEAWAYS

- 1 CloudWatch is part of the solution, not the full strategy.
- 2 Use metrics, logs, traces and application telemetry.
- 3 Design for high-cardinality where needed.
- 4 Build meaningful dashboards and alarms.
- 5 Continuously look for blind spots.
- 6 Observability is about understanding the system, not just collecting data.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Debugging AWS From the Evidence Outward

Define blast radius, correlate changes, inspect metrics and logs, test hypotheses, isolate the failure domain, mitigate, verify.

THE DEBUGGING PROCESS



1. DEFINE BLAST RADIUS

- 1 Identify what is failing (symptoms).
- 2 Determine affected services and users.
- 3 Map dependencies (upstream and downstream).
- 4 Check if the issue is regional, AZ-specific or account-specific.
- 5 Understand business impact (what is critical).

2. CORRELATE CHANGES

- 1 Check recent deployments.
- 2 Review infrastructure changes (Terraform, CloudFormation).
- 3 Look at configuration updates.
- 4 Check for scaling events.
- 5 Investigate recent AWS service health events.

3. INSPECT METRICS AND LOGS

- 1 CloudWatch metrics (service, application).
- 2 CloudWatch Logs (application, system).
- 3 VPC Flow Logs (network issues).
- 4 ALB / NLB logs (request and patterns).
- 5 RDS / DynamoDB / S3 / Lambda logs.

4. TEST HYPOTHESES

- 1 Form a hypothesis based on evidence.
- 2 Validate using metrics, logs and traces.
- 3 Eliminate what is not the cause.
- 4 Use targeted tests (e.g. curl, AWS CLI, traceroute, describe commands).
- 5 Refine the hypothesis until the root cause is clear.

5. ISOLATE FAILURE DOMAIN

- 1 Identify the exact component failing (e.g. EC2, RDS, API, network).
- 2 Determine if the issue is internal or external dependency.
- 3 Check for cascading failures.
- 4 Isolate to an AZ, VPC, subnet, service or account.
- 5 Confirm the root cause with evidence.

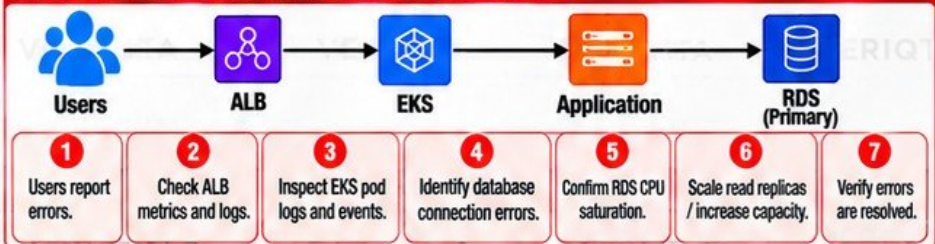
6. MITIGATE

- 1 Apply the least risky and fastest mitigation.
- 2 Roll back changes if needed.
- 3 Scale, failover or switch to a healthy resource.
- 4 Apply temporary fixes (e.g. disable a feature).
- 5 Communicate status to stakeholders.

7. VERIFY

- 1 Confirm the service is working as expected.
- 2 Check metrics, logs and user experience.
- 3 Validate that errors, latency and saturation have returned to normal.
- 4 Monitor for a period of time.
- 5 Document the findings and root cause.

EXAMPLE: DEBUGGING HIGH ERROR RATE



USEFUL AWS TOOLS AND DATA SOURCES

	CloudWatch	Metrics, logs, alarms, dashboards
	AWS X-Ray	Distributed tracing, service map
	CloudTrail	API activity, changes
	AWS Config	Configuration changes, compliance
	VPC Flow Logs	Network traffic (accepted/rejected)
	ELB / ALB / NLB Logs	Request logs, error codes
	RDS / Database Logs	Slow queries, errors, connections
	EKS Logs	Control plane and application logs
	AWS Health	Service health events
	SNS / Incident Tools	Notifications and communication

COMMON FAILURE AREAS

- 1 Networking (VPC, security groups, NACLs, DNS, NAT).
- 2 Load balancers (target failures, health checks).
- 3 Compute (EC2, Auto Scaling, EKS nodes).
- 4 Databases (RDS, Aurora, DynamoDB).
- 5 Storage (S3, EFS, EBS).
- 6 IAM and permissions.
- 7 Service quotas and throttling.
- 8 External dependencies (third-party APIs).
- 9 Configuration and environment variables.
- 10 Application code and deployment issues.

KEY TAKEAWAYS

- ✓ Start with evidence, not assumptions.
- ✓ Always consider blast radius.
- ✓ Correlate changes with timelines.
- ✓ Use multiple data sources (metrics, logs, traces).
- ✓ Test and validate hypotheses.
- ✓ Isolate the failure domain.
- ✓ Mitigate quickly and safely.
- ✓ Verify and monitor after the fix.
- ✓ Document what happened.
- ✓ Build runbooks for common failures.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

The AWS Deployment That Passed but Broke Production

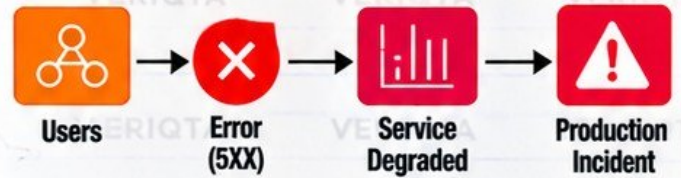
Pipeline success versus service health, configuration errors, dependency failures, rollout signals, rollback criteria, and verification.

1. PIPELINE PASSED



- ✓ Code built successfully.
- ✓ All automated tests passed.
- ✓ Artifact created and deployed.
- ✓ Pipeline reported success.

2. BUT PRODUCTION BROKE



- ✗ Deployment succeeded but service was unhealthy.
- ✗ Users started seeing errors (e.g. 5XX).
- ✗ Key functionality stopped working.
- ✗ Alerts triggered after deployment.
- ✗ Rollback was required.

3. COMMON ROOT CAUSES

- 1 Configuration errors (wrong env vars, feature flags, IAM permissions).
- 2 Dependency failures (database, API, queue, external service).
- 3 Incompatible changes (schema, version, protocol).
- 4 Partial deployment (some instances updated, others not).
- 5 Infrastructure changes (security groups, load balancer, routing).
- 6 Missing or insufficient health checks.
- 7 Caching issues (old data, invalidation).
- 8 Resource limits (CPU, memory, connections).

4. ROLLOUT SIGNALS

- 1 Increased error rates (4XX/5XX).
- 2 Higher latency.
- 3 Drop in successful transactions.
- 4 Health check failures.
- 5 Increased throttling.
- 6 User complaints.
- 7 Anomalies in metrics and logs.
- 8 External dependency errors.

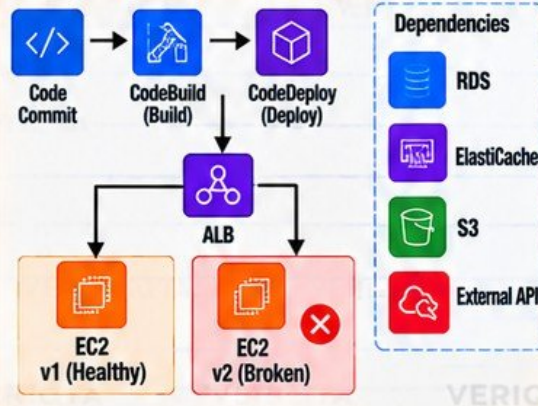
5. ROLLBACK CRITERIA

- 1 Error rate above threshold.
- 2 Critical functionality not working.
- 3 Latency significantly degraded.
- 4 Health checks failing.
- 5 Increased rollback-related alerts.
- 6 Data corruption risk.
- 7 Business impact to users.
- 8 No quick fix available.

6. VERIFICATION AFTER ROLLBACK

- 1 Confirm previous version is running.
- 2 Validate service health (metrics, logs, traces).
- 3 Check dependencies are healthy.
- 4 Monitor error rates and latency.
- 5 Verify user functionality end-to-end.
- 6 Confirm alerts have cleared.
- 7 Keep monitoring for a sustained period.
- 8 Document the incident and root cause.

7. EXAMPLE ARCHITECTURE



8. KEY TAKEAWAYS

- ✓ A passing deployment does not mean a healthy service.
- ✓ Monitor real user impact, not just pipeline status.
- ✓ Use proper health checks and canary/blue-green strategies.
- ✓ Define clear rollback criteria before deployment.
- ✓ Validate dependencies and configuration.
- ✓ Always verify after rollback.
- ✓ Document and learn from the incident.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

AWS Quotas, Throttling, and Invisible Capacity Ceilings

API limits, service quotas, connection ceilings, scaling assumptions, quota monitoring, capacity planning, and failure prevention.

1. WHAT ARE AWS QUOTAS?

- 1 Service quotas limit how much of an AWS resource you can use.
- 2 They exist to ensure stability, fair usage and regional capacity.
- 3 Quotas can be account-level, region-level or service-level.
- 4 Exceeding a quota can cause throttling, API errors or resource creation failures.
- 5 Many quotas are adjustable (via Service Quotas), but some have hard limits.



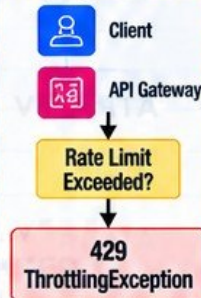
2. COMMON QUOTAS AND LIMITS (EXAMPLES)

Service	Quota / Limit	Default Limit (Example)	Impact if Exceeded
EC2	Running instances per region	Varies (e.g. 20)	Cannot launch more instances
EKS	Clusters per region	Varies (e.g. 10)	Cluster creation fails
VPC	VPCs per region	Varies (e.g. 5)	Cannot create more VPCs
Lambda	Concurrent executions	Varies (e.g. 1,000)	Throttling (429 errors)
RDS	DB instances per region	Varies (e.g. 40)	Cannot create more DB instances
API Gateway	Requests per second (API rate)	Varies	Throttling (429 errors)
S3	Requests per second (per prefix)	High but limited	Slowdowns or throttling

3. THROTTLING AND API LIMITS

- 1 When you exceed API rate limits, AWS returns ThrottlingException or HTTP 429 errors.
- 2 Throttling can happen at the account, region or resource level.
- 3 It can affect automated deployments, scaling events and high-traffic applications.
- 4 Implement exponential backoff and retry logic.
- 5 Monitor API usage to detect throttling early.

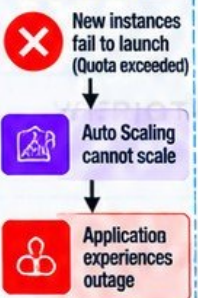
API REQUEST FLOW



4. WHERE QUOTAS CAN CAUSE FAILURES

- 1 Auto Scaling cannot launch new instances.
- 2 EKS cannot register new nodes.
- 3 Lambda functions get throttled.
- 4 RDS cannot create read replicas.
- 5 API Gateway or other APIs return 429 errors.
- 6 NAT Gateway hits connection limits.
- 7 IAM role or policy limits block new resources.
- 8 Unexpected quota limits during a regional expansion.

REAL EXAMPLE



5. MONITORING QUOTAS

- 1 Use AWS Service Quotas to view current limits.
- 2 Set up CloudWatch alarms for quota usage (where available).
- 3 Monitor API throttling metrics (e.g. Throttles, 4XX errors).
- 4 Use AWS Trusted Advisor for quota recommendations.
- 5 Get notified before you hit critical limits.



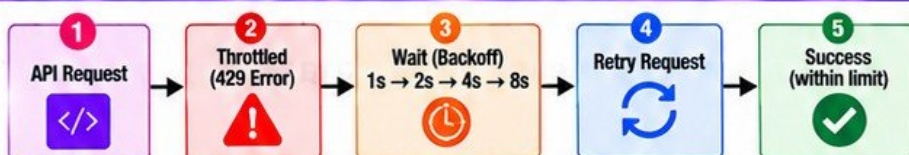
6. CAPACITY PLANNING BEST PRACTICES

- 1 Understand default quotas for your key services.
- 2 Request quota increases early (before you need them).
- 3 Design architectures with quotas in mind (multi-AZ, multi-region).
- 4 Test scaling scenarios. (e.g. load testing).
- 5 Monitor and set alarms.
- 6 Document quota dependencies.
- 7 Review quotas regularly as you grow.

7. KEY TAKEAWAYS

- ✓ Quotas are real and can cause production failures.
- ✓ Always check service quotas during design and scaling.
- ✓ Monitor API usage and throttling.
- ✓ Request quota increases early.
- ✓ Implement retry logic with exponential backoff.
- ✓ Include quotas in your capacity planning and runbooks.
- ✓ Regularly review quotas as your environment grows.

EXAMPLE: HANDLING THROTTLING WITH EXPONENTIAL BACKOFF



Example (Python - Exponential Backoff)

```

import time
import random
for attempt in range(5):
    try:
        response = call_aws_api()
        break # success
    except ThrottlingException:
        wait = (2 ** attempt) + random.uniform(0, 1)
        time.sleep(wait)
  
```



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



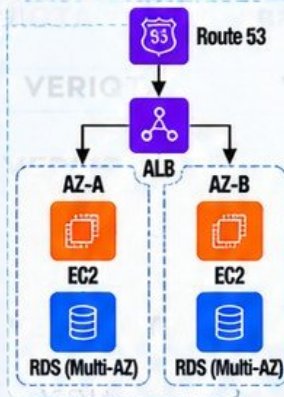
@veriqta

The Senior AWS Production Readiness Review

Final architecture review across failure domains, IAM, networking, capacity, observability, deployment safety, recovery, cost, security, quotas, and operational ownership.

1. ARCHITECTURE & FAILURE DOMAINS

- ✓ Multi-AZ design for critical components.
- ✓ No single points of failure (SPOF).
- ✓ Clearly defined failure domains (AZ, Region, service).
- ✓ Dependency mapping (internal and external).
- ✓ Resilience patterns (redundancy, retries, circuit breaking).



2. IAM & ACCESS MANAGEMENT

- ✓ Least privilege access.
- ✓ Proper use of roles (no long-lived keys).
- ✓ Resource-level permissions.
- ✓ MFA enabled for all humans.
- ✓ Regular access reviews and cleanup.
- ✓ Cross-account access controlled.



IAM

3. NETWORKING & CONNECTIVITY

- ✓ VPC design (public/private subnets).
- ✓ Proper routing and NACLs.
- ✓ Secure connectivity (VPN / Direct Connect).
- ✓ Egress control (NAT, VPC endpoints).
- ✓ DNS configuration and health checks.
- ✓ No unintended public exposure.



VPC

4. CAPACITY & SCALING

- ✓ Right instance types and sizing.
- ✓ Auto Scaling configured and tested.
- ✓ Load balancing across multiple AZs.
- ✓ Service quotas reviewed and sufficient.
- ✓ Performance testing under expected load.
- ✓ Headroom for traffic growth and spikes.



5. OBSERVABILITY & MONITORING

- ✓ Metrics, logs and traces enabled.
- ✓ Meaningful dashboards and alerts.
- ✓ Application-level telemetry.
- ✓ Distributed tracing (X-Ray / OpenTelemetry).
- ✓ Alerting on key business and infrastructure metrics.
- ✓ Regular review of blind spots.



CloudWatch

6. DEPLOYMENT SAFETY

- ✓ CI/CD pipeline with automated tests.
- ✓ Safe deployment strategy (blue/green, canary).
- ✓ Rollback criteria defined and tested.
- ✓ Feature flags for risky changes.
- ✓ Configuration management and version control.
- ✓ Post-deployment verification (metrics, logs, health checks).



7. RECOVERY & DISASTER PREPAREDNESS

- ✓ Backups enabled and tested (restore).
- ✓ Multi-AZ and/or multi-region strategy.
- ✓ RTO and RPO defined.
- ✓ Data replication for critical services.
- ✓ Disaster recovery runbooks.
- ✓ Regular DR testing and validation.



8. COST & EFFICIENCY

- ✓ Right sizing and cost optimization.
- ✓ Use of Savings Plans / Reserved Instances.
- ✓ Idle resources removed.
- ✓ Cost monitoring and budgets.
- ✓ Tagging strategy for cost allocation.
- ✓ Regular cost reviews.



9. SECURITY & COMPLIANCE

- ✓ Security groups and NACLs properly configured.
- ✓ Encryption at rest and in transit.
- ✓ Secrets management (AWS Secrets Manager).
- ✓ Vulnerability scanning and patching.
- ✓ Compliance requirements met (e.g. SOC 2, ISO, PCI).
- ✓ Regular security reviews and audits.



10. QUOTAS & LIMITS

- ✓ Service quotas reviewed (EC2, RDS, EKS, etc.).
- ✓ API rate limits and throttling considered.
- ✓ Connection limits (NAT, ALB, databases).
- ✓ Monitoring for quota usage.
- ✓ Request quota increases early.
- ✓ Runbooks for quota-related failures.



11. OPERATIONAL OWNERSHIP

- ✓ Clear ownership and on-call model.
- ✓ Runbooks for common failures.
- ✓ Incident response process defined.
- ✓ Regular game days and failure testing.
- ✓ Documentation up to date.
- ✓ Knowledge transfer and team readiness.



KEY TAKEAWAYS

- ✓ A production-ready system is resilient, observable, secure and cost-efficient.
- ✓ Review every layer, not just the application.
- ✓ Test your assumptions.
- ✓ Eliminate single points of failure.
- ✓ Validate recovery and rollback procedures.
- ✓ Continuously monitor, improve and review.

PRODUCTION READINESS CHECKLIST (QUICK VIEW)

Architecture & Failure Domains	IAM & Access	Networking & Connectivity	Capacity & Scaling	Observability & Monitoring	Deployment Safety	Recovery & DR	Cost & Efficiency	Security & Compliance	Quotas & Limits	Operational Ownership